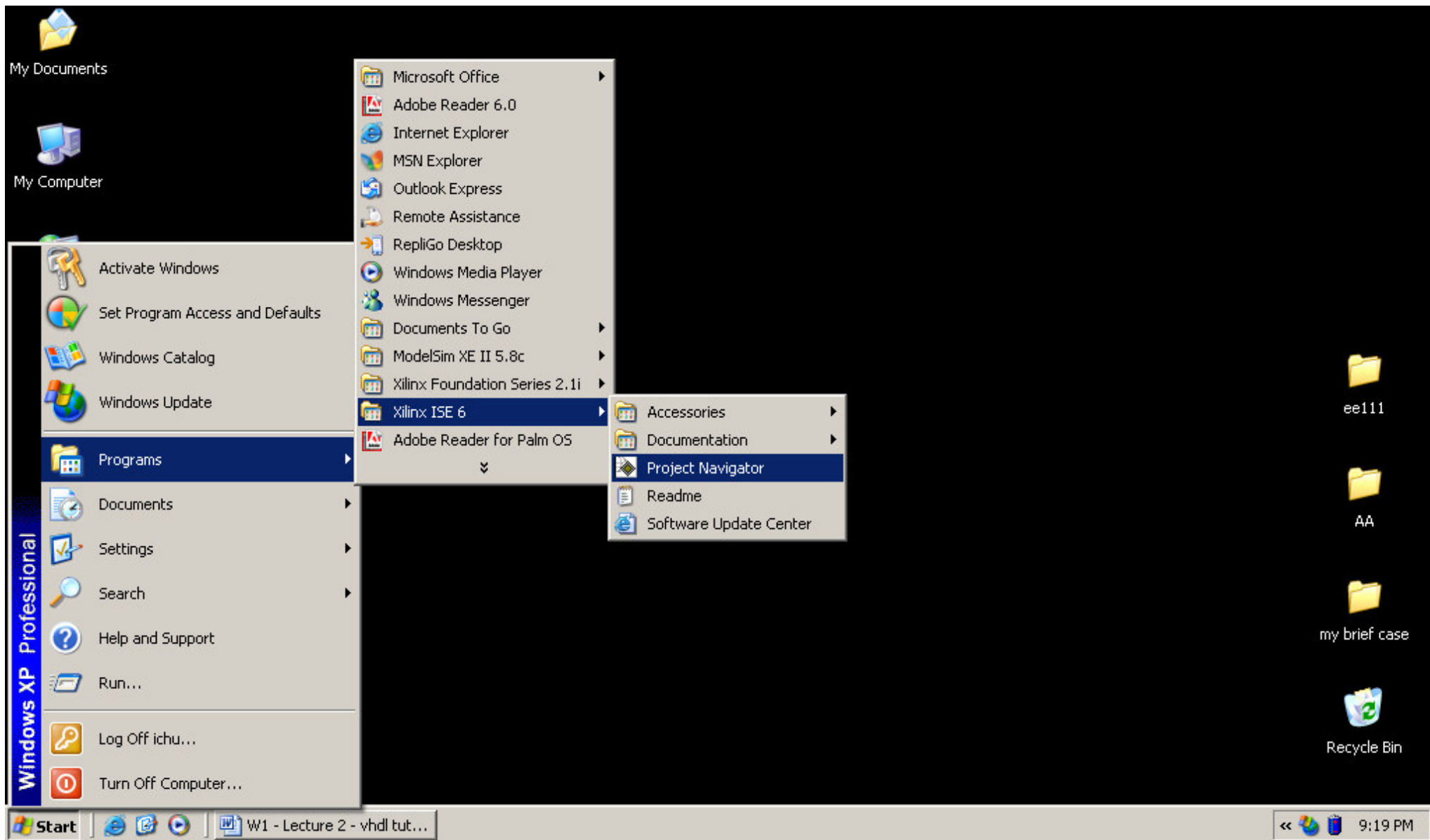
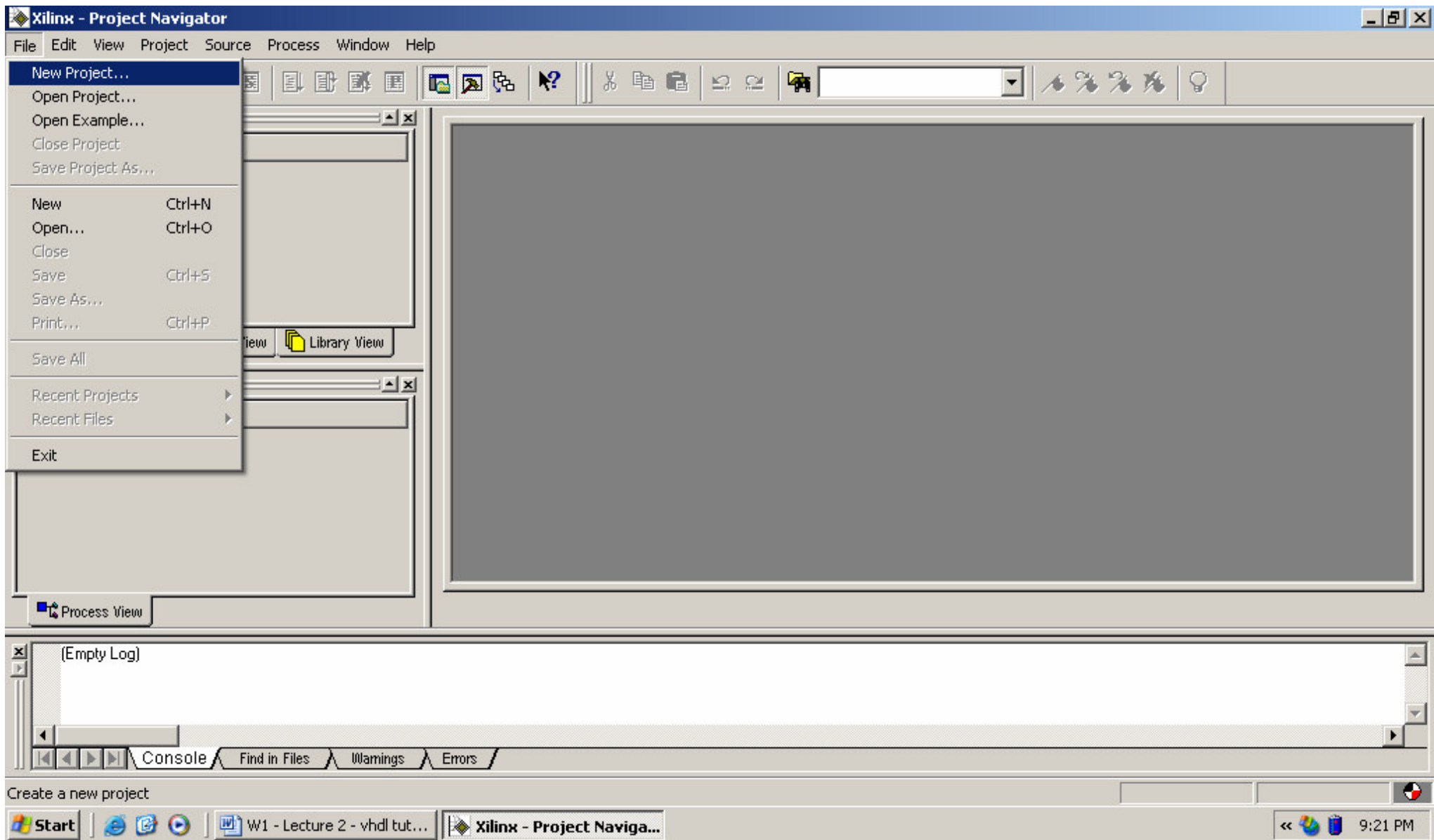


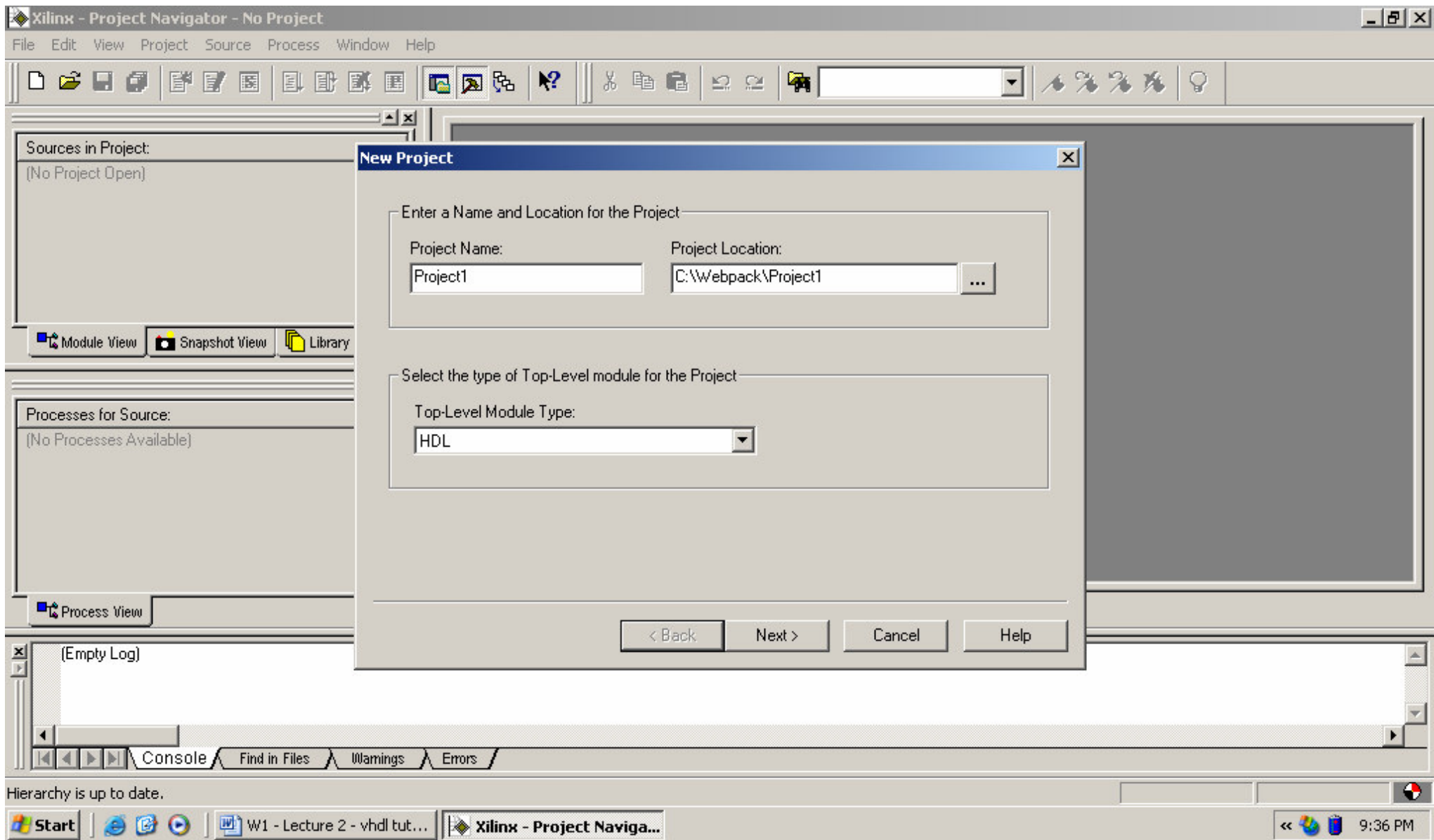
VHDL TUTORIAL using Xilinx's WEBPACK and ModelSim



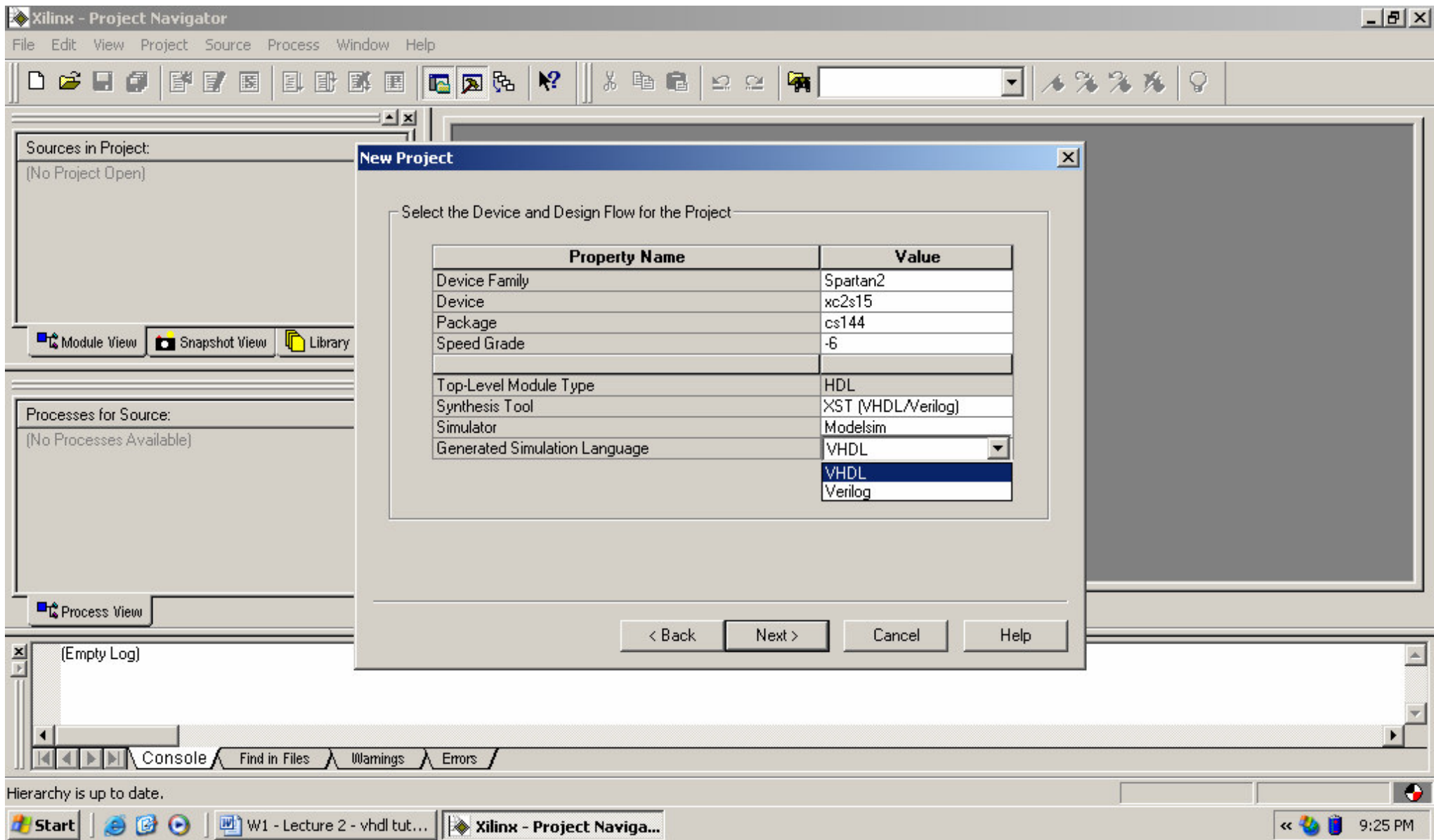
After you have installed the Xilinx's WebPack and ModelSim, start the Xilinx ISE 6 > Project Navigator



You will come to the blank project (next time will have the most-recent project loaded by default).
Click on File > New Project to start a new project

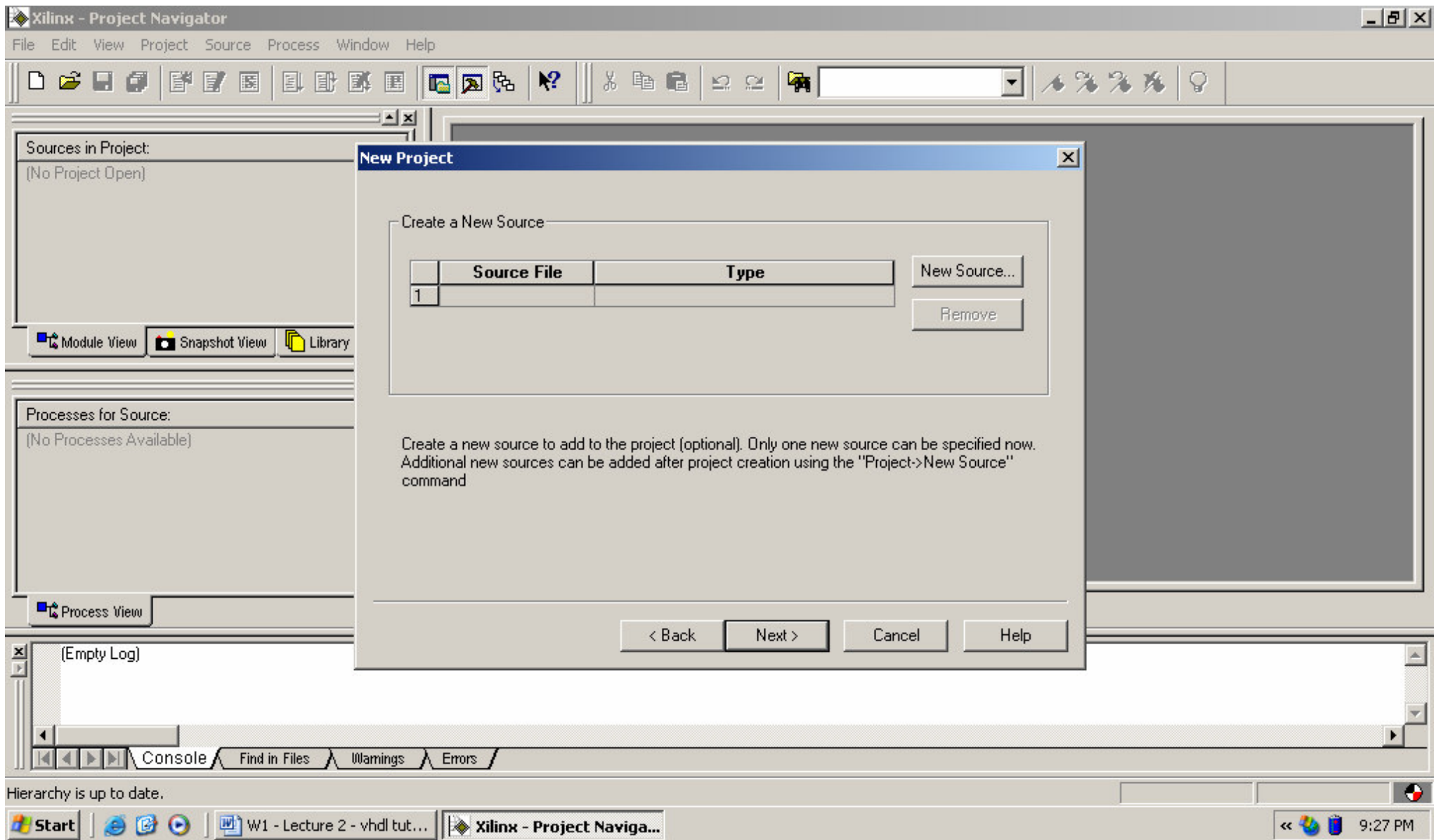


Be sure to select **HDL** as the top-level module type.
You can name your project, but let's keep it simple and called this "Project1".

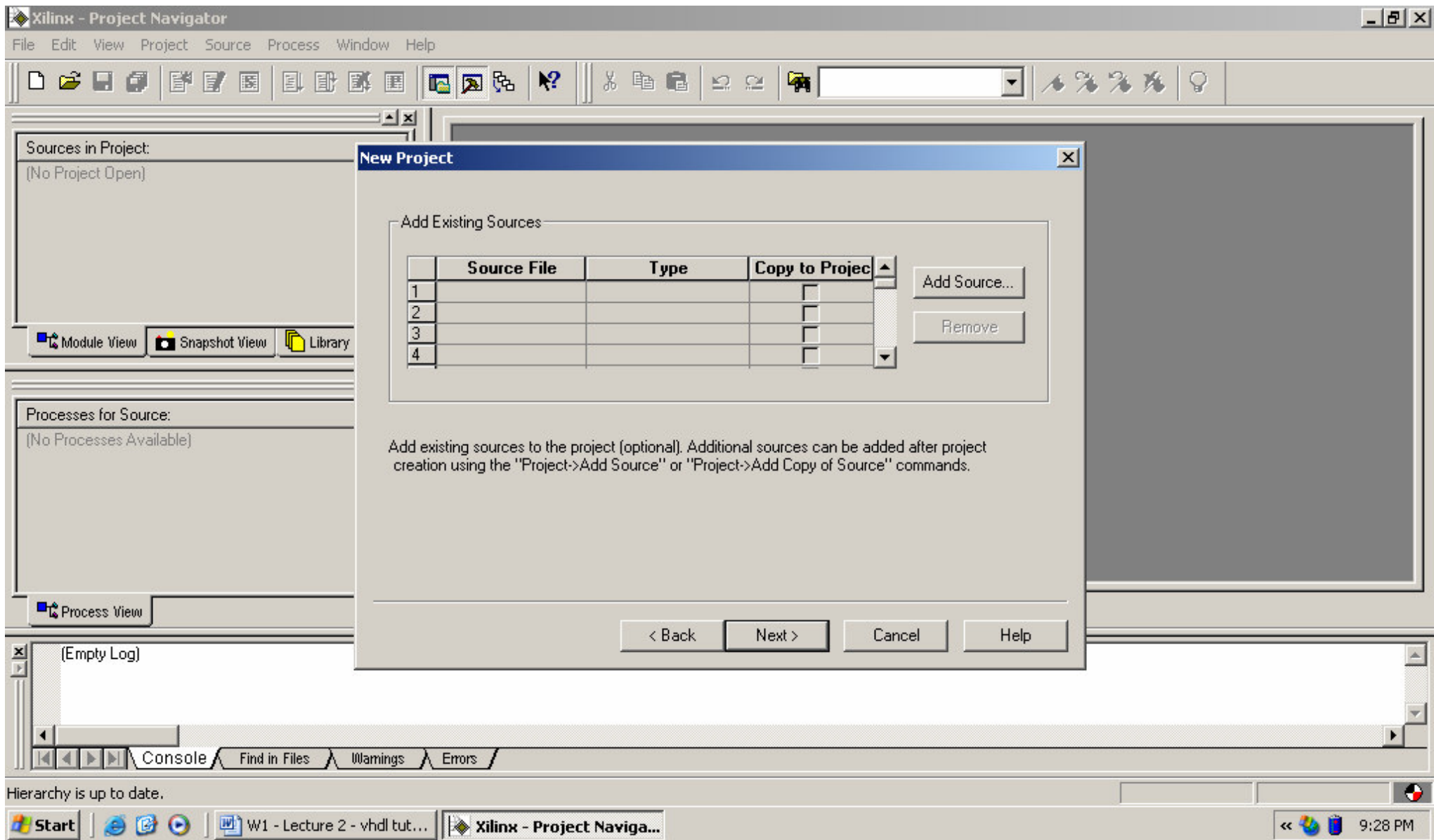


Be sure to select VHDL for “Generated Simulation Language”. Leave everything else as default.

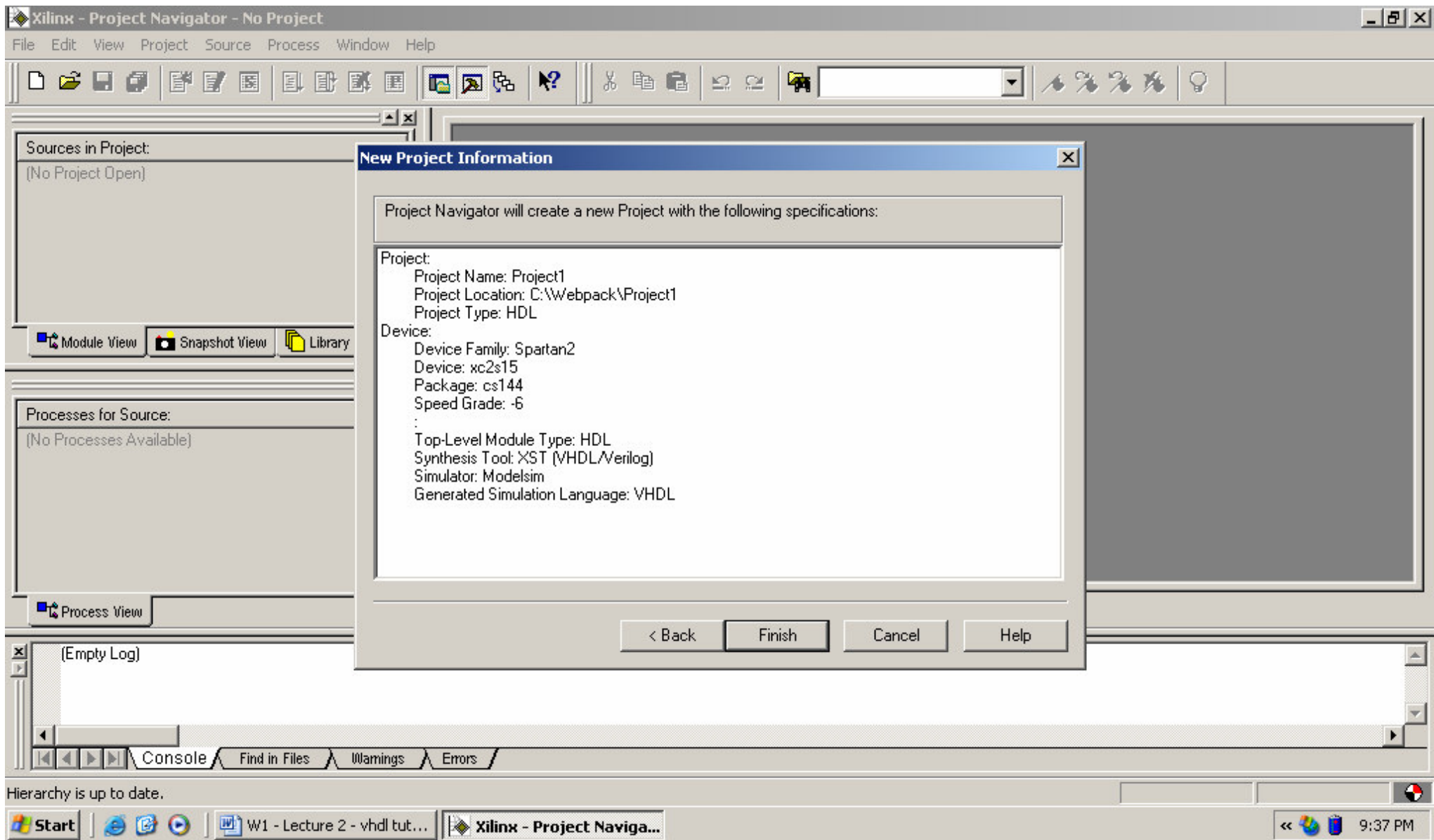
There are Device family & Device on first two row. These are the actual hardware your VHDL will map to as the final product. You can buy these chips, but they can be costly (hundreds to thousands of dollars depending on what you select). Note, the tool is free, but hardware is not and that's how Xilinx makes money.



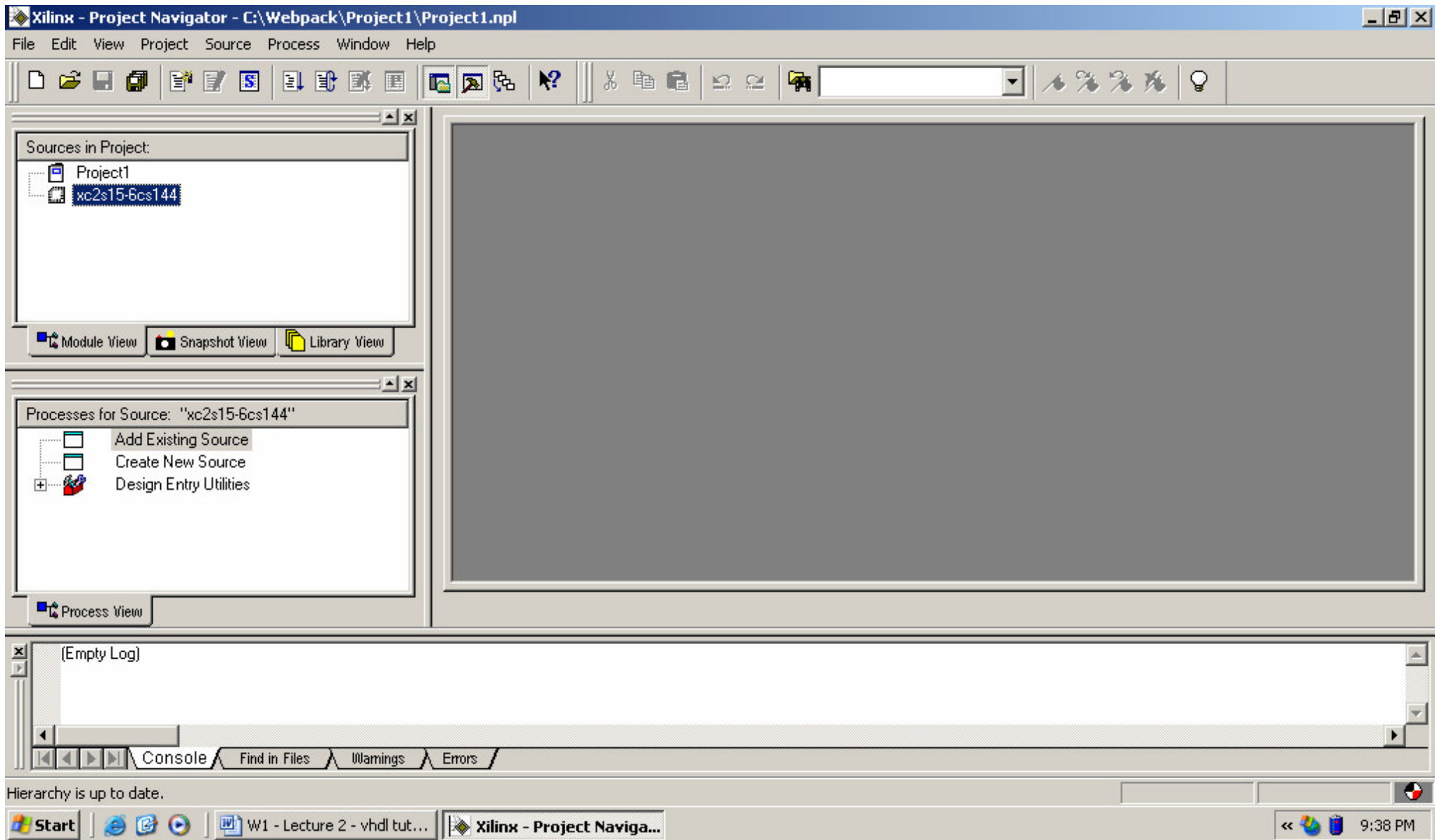
Leave everything as default because you are starting with no files. You may add new source file later also.



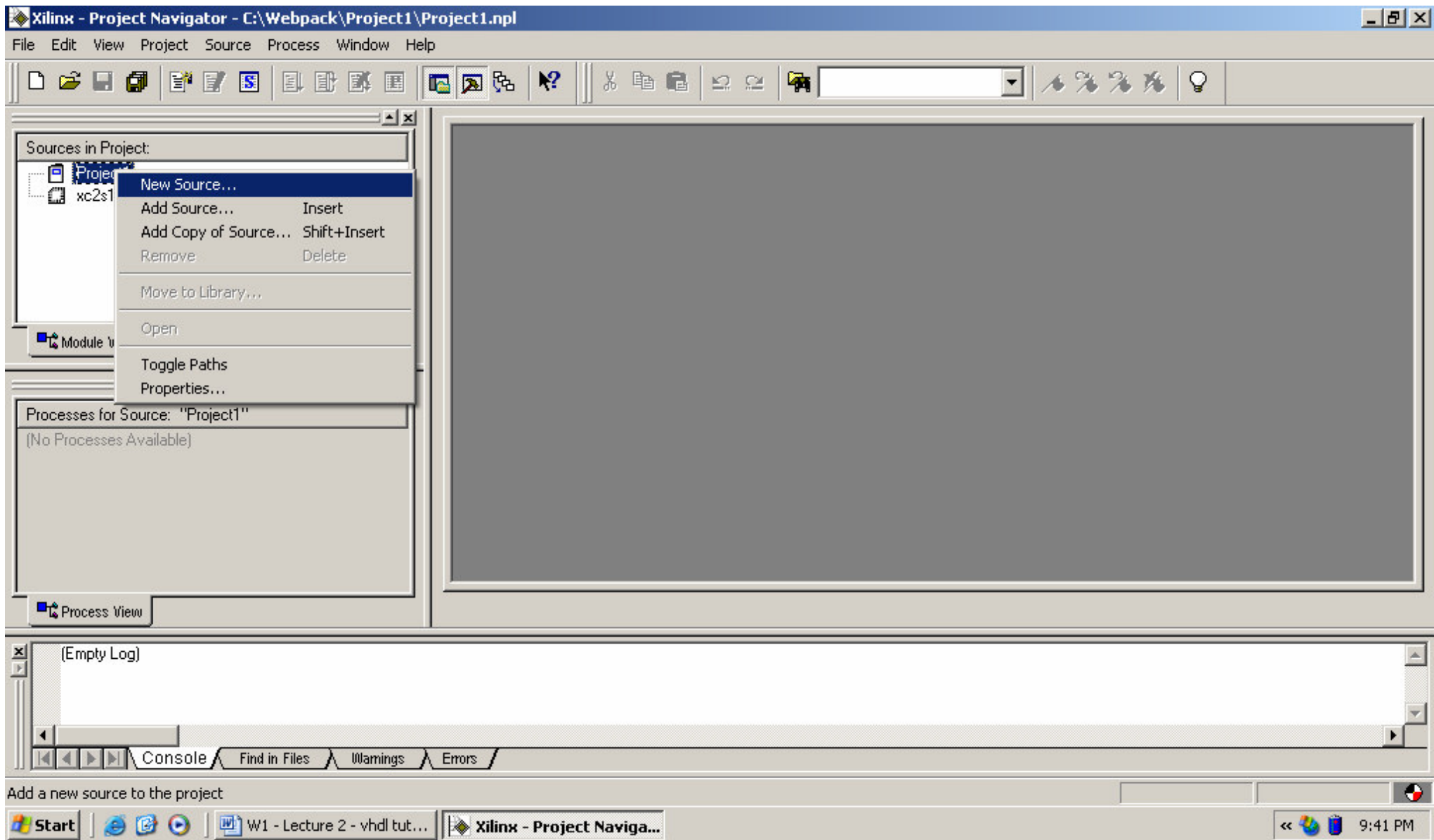
Leave everything as default because you are starting with no files. You may add existing files later as well.



Click Finish to create the new project.

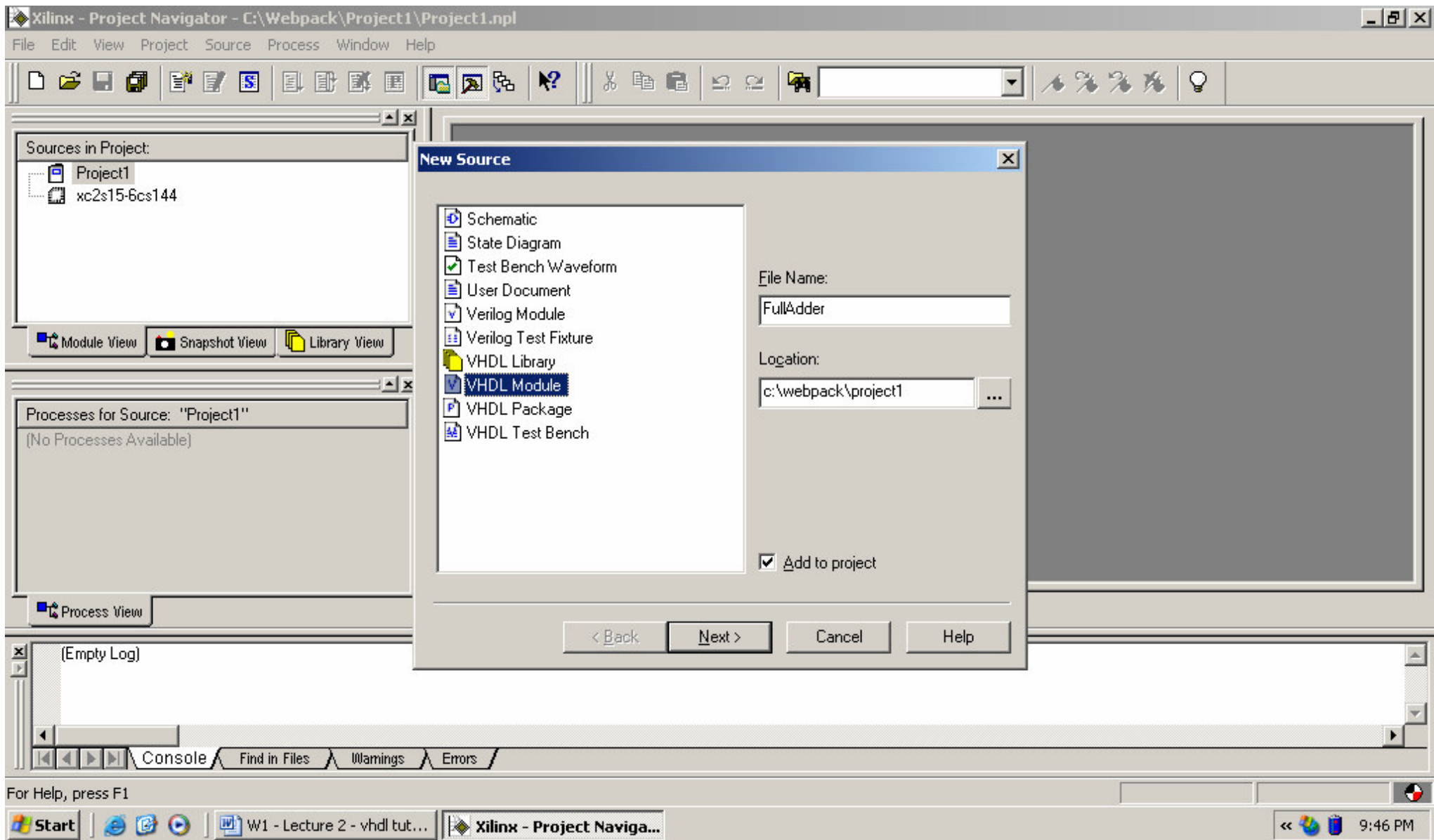


This is the environment without any VHDL Files

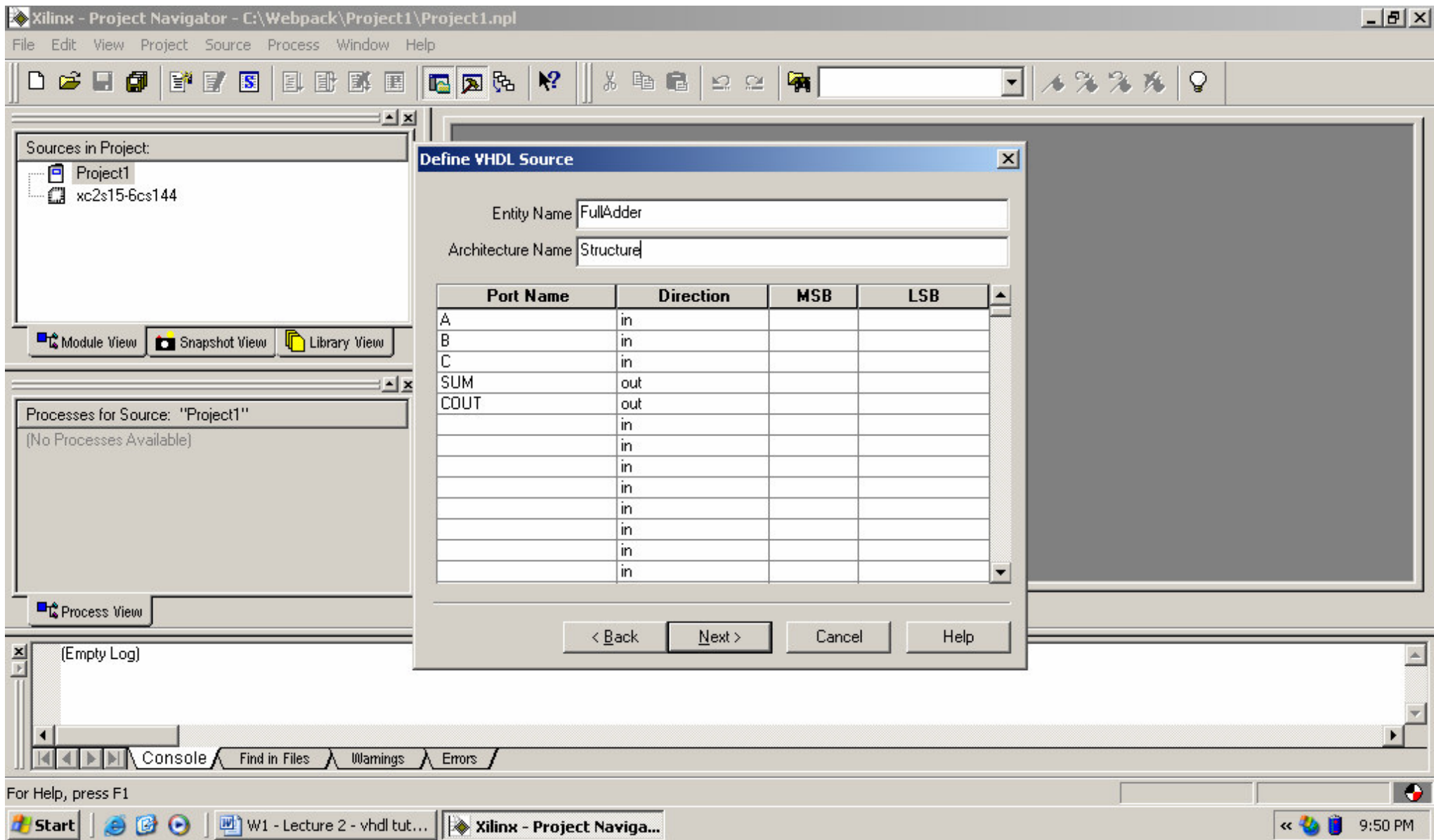


Now, let's create a full-adder using VHDL.

Note, you can now add existing source (a pointer) or add Copy of Source (create a new file from the source). Source is usually a .vhd file. If your partner finished one vhd1 and gives you the file, you want to select add Copy of Source. You can always Ctrl-c Ctrl-v

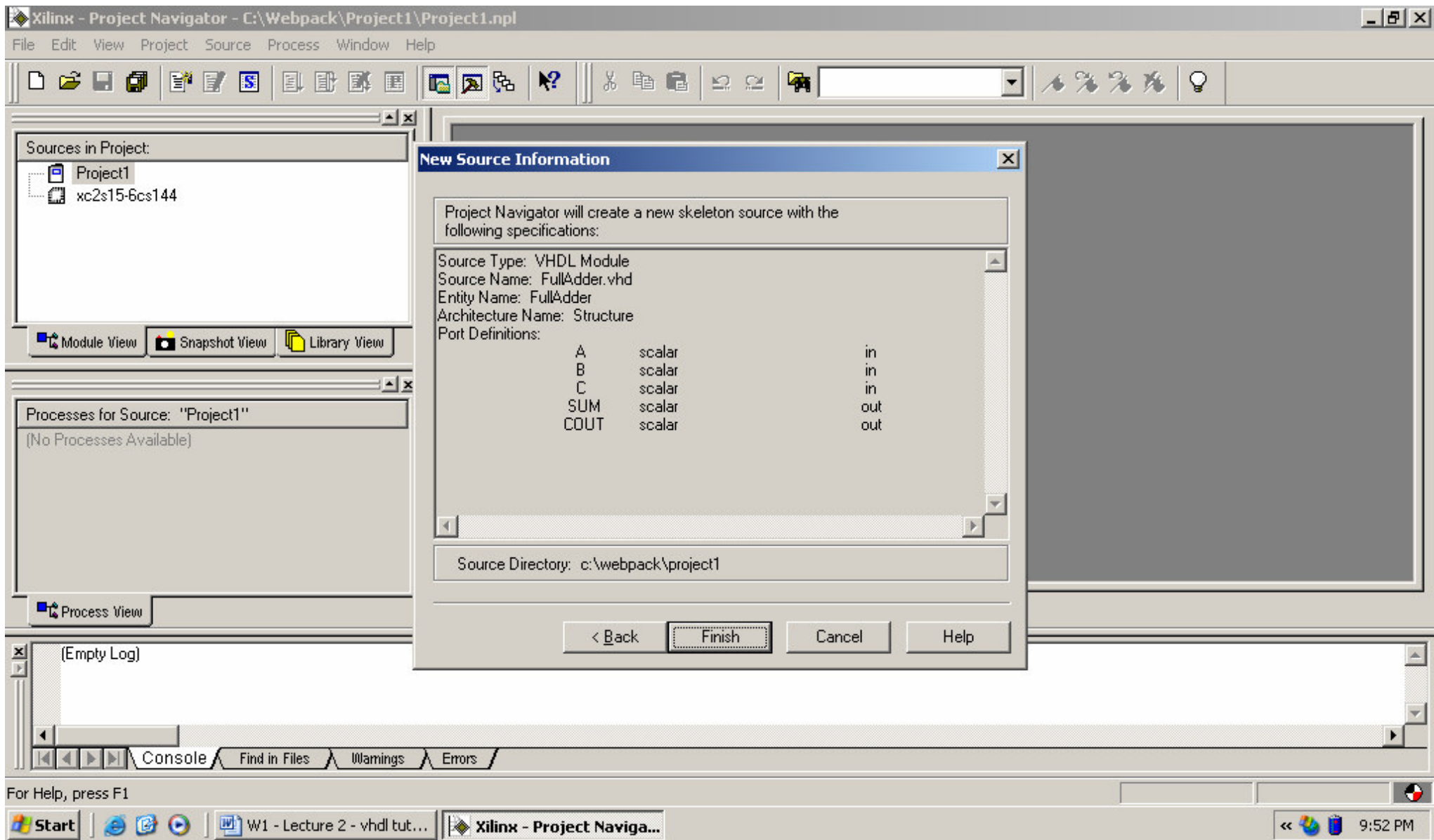


Be sure to select VHDL Module

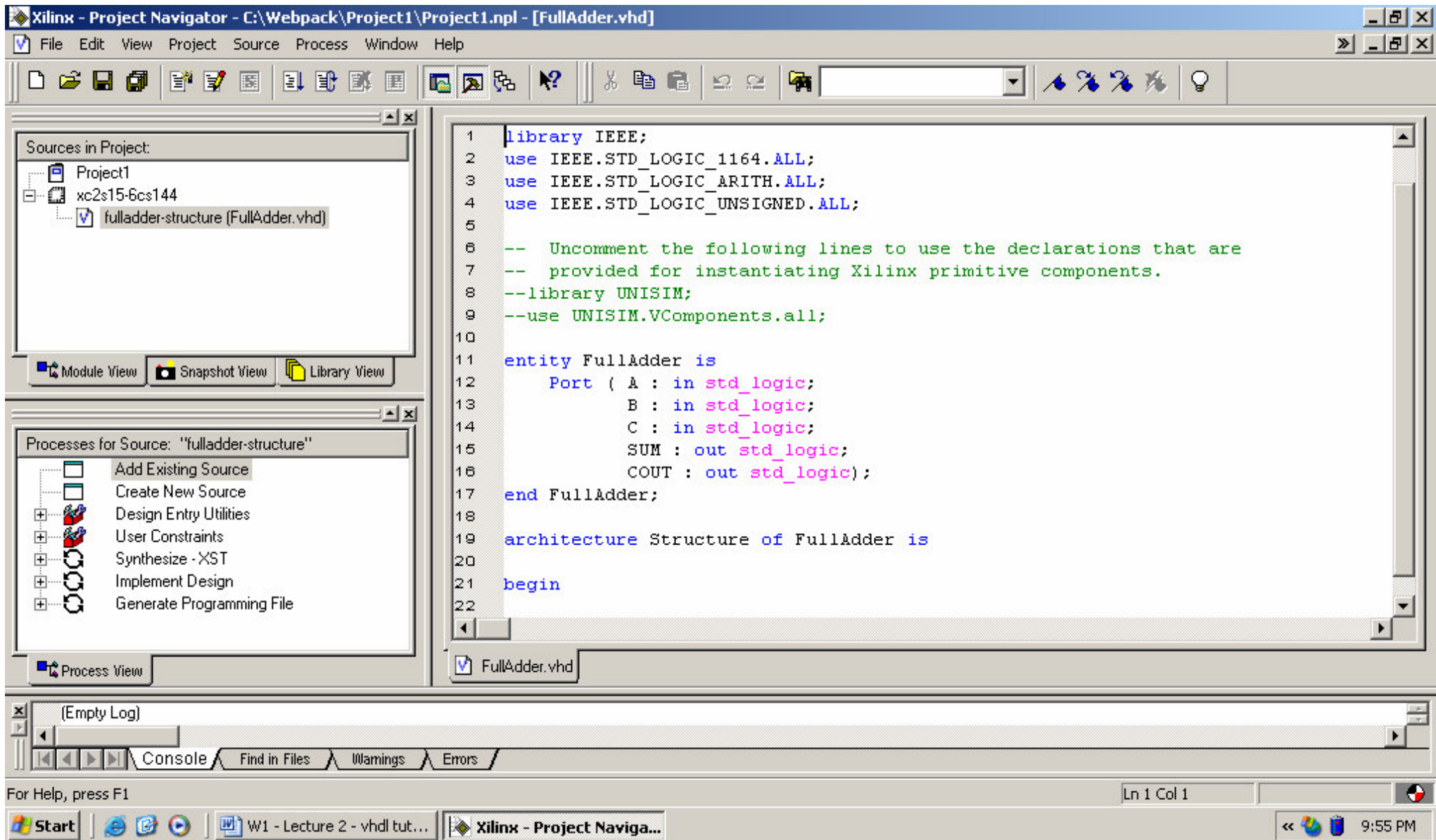


Architecture Name: Structure (concurrent statement).

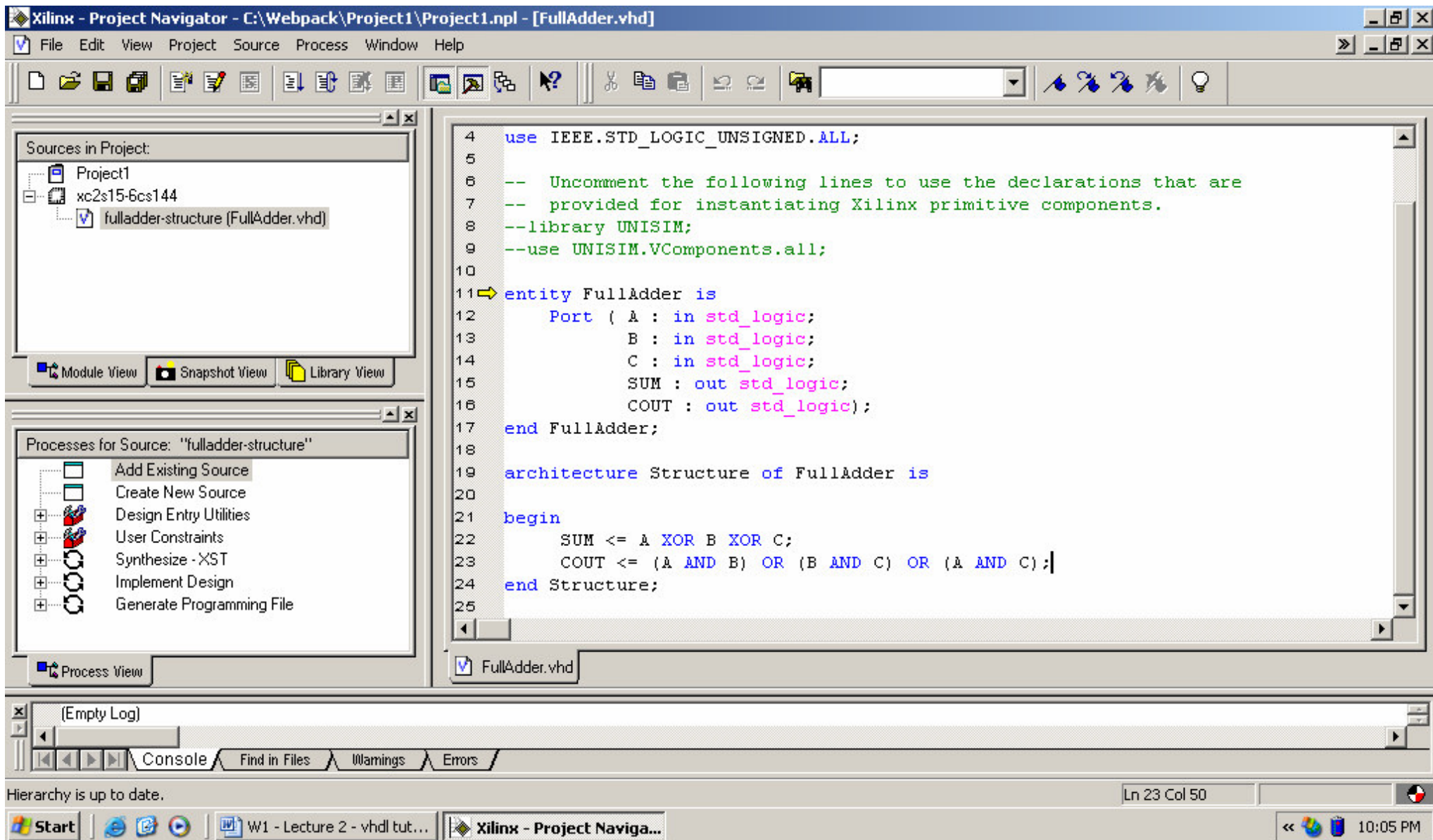
Architecture Name can also be Behavioral if it's a sequential process like while loop and if-else statements



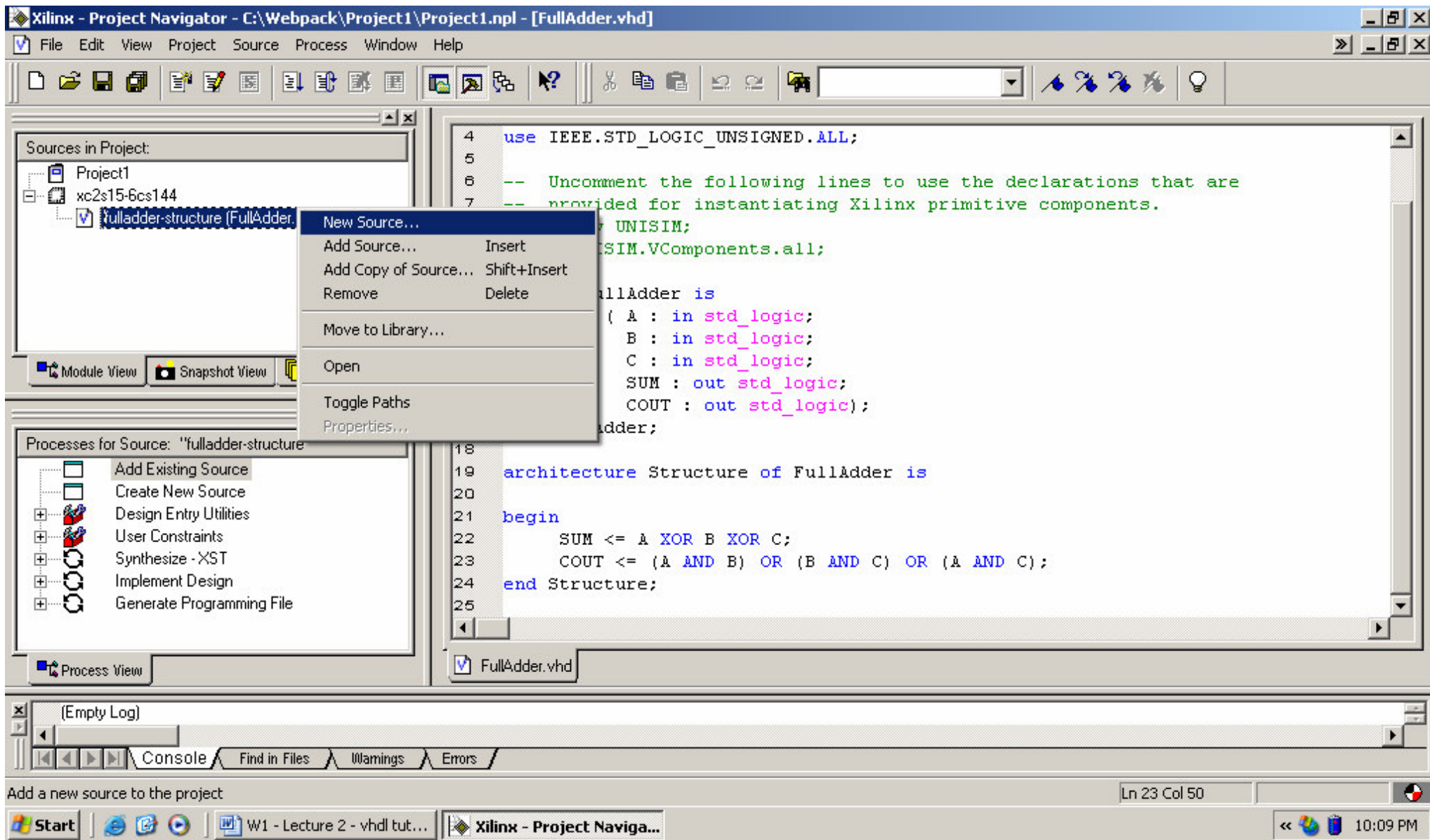
It shows a quick summary of what you are about to create, click “Finish”.



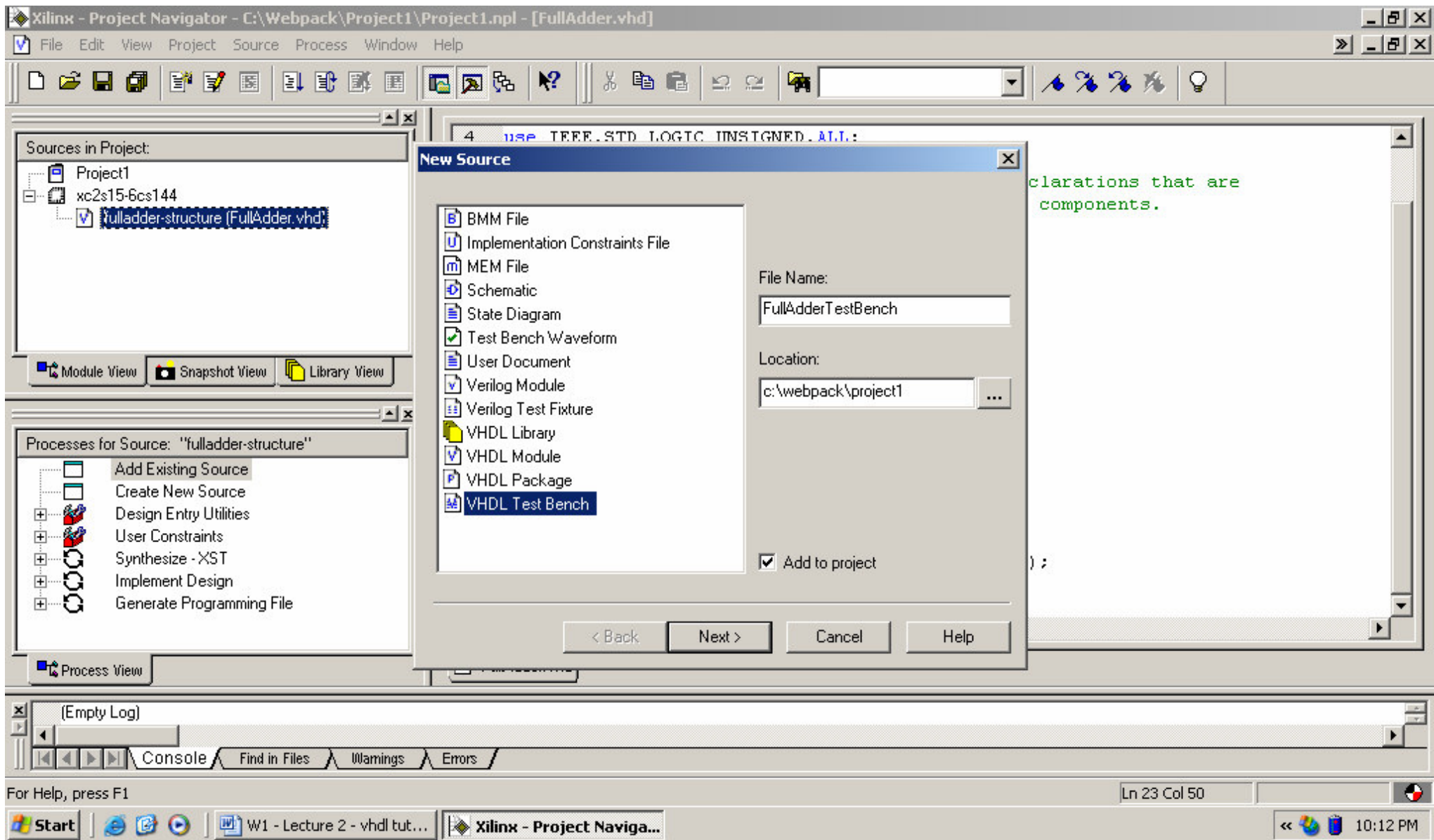
Some code will be automatically generated (libraries, declarations, and skeleton of the VHDL code)



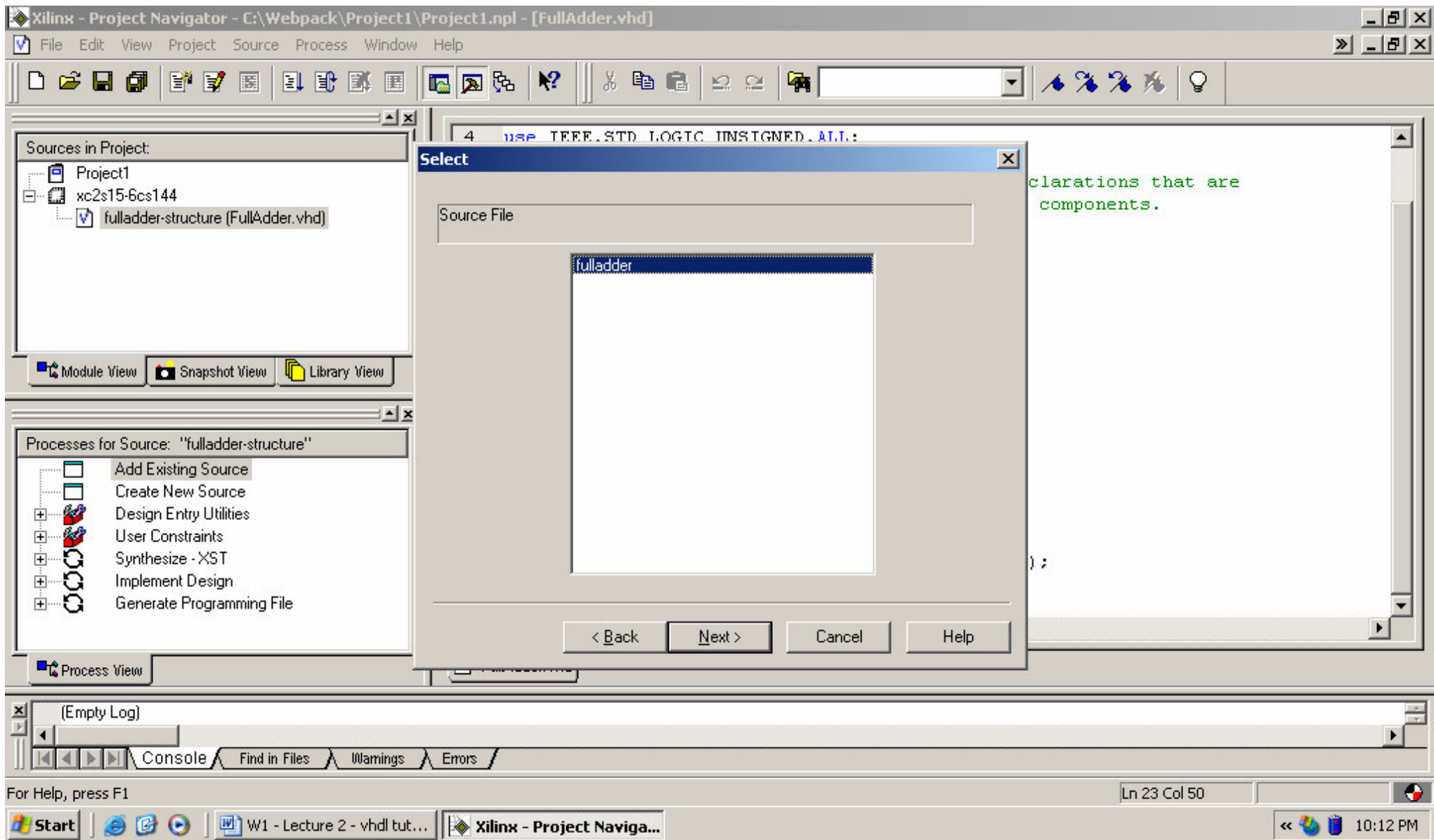
Add in the VHDL code for full adder logic



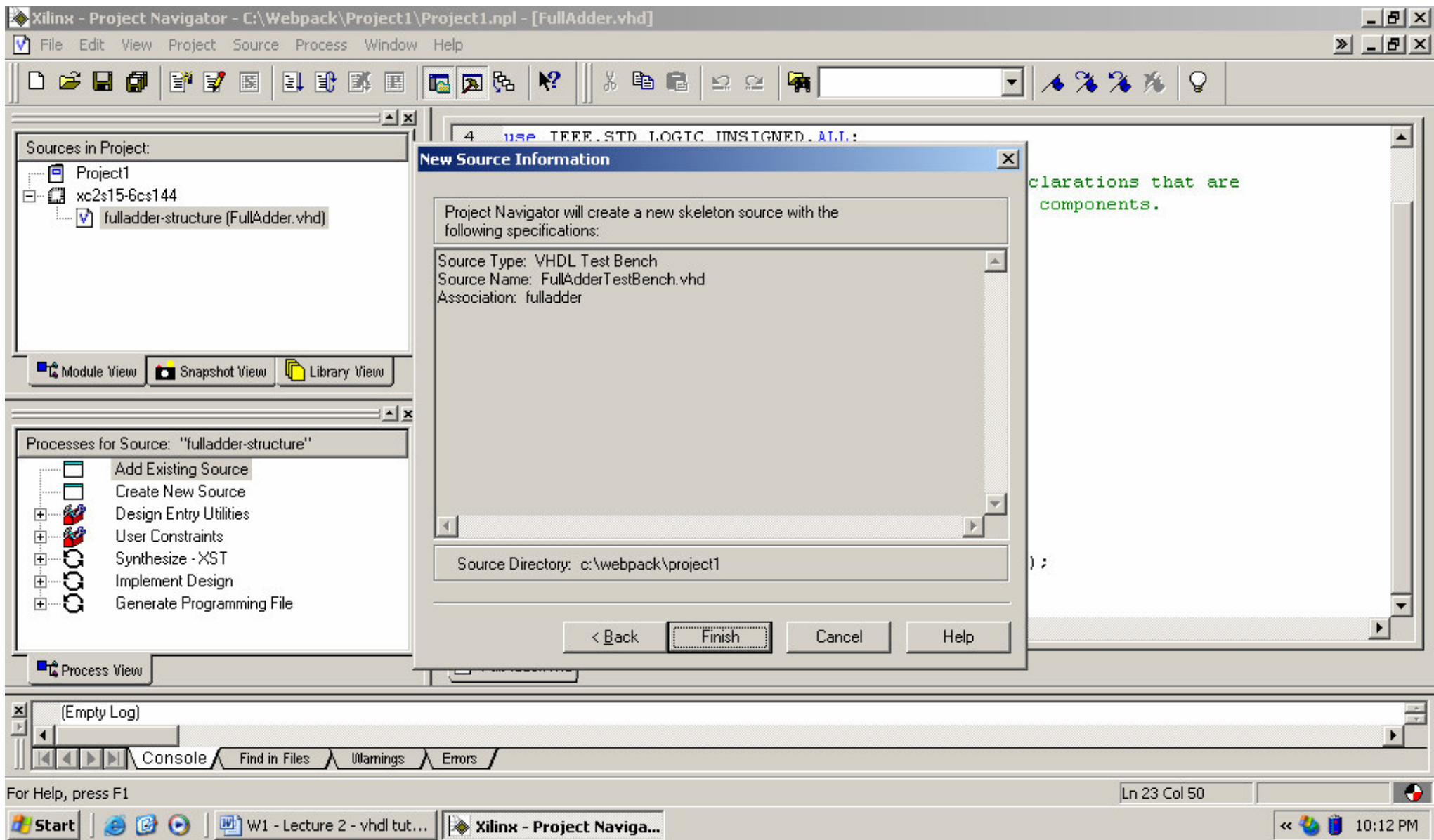
Now, you can create a test bench for the full-adder to test whether the logic is what you expected.
(Be sure to click on FullAdder VHDL file and right click)



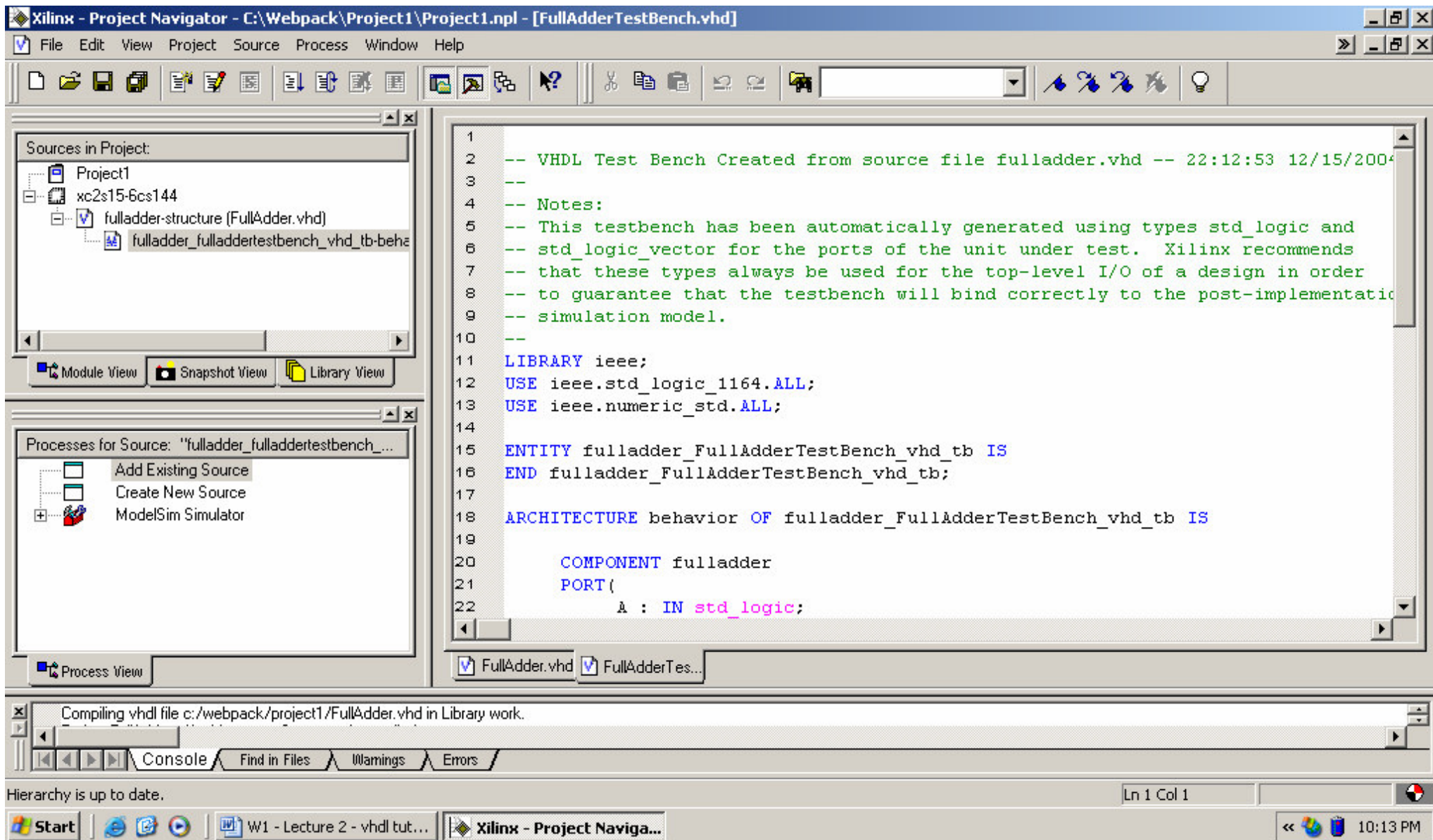
Be sure to select VHDL Test Bench and give it a file name.



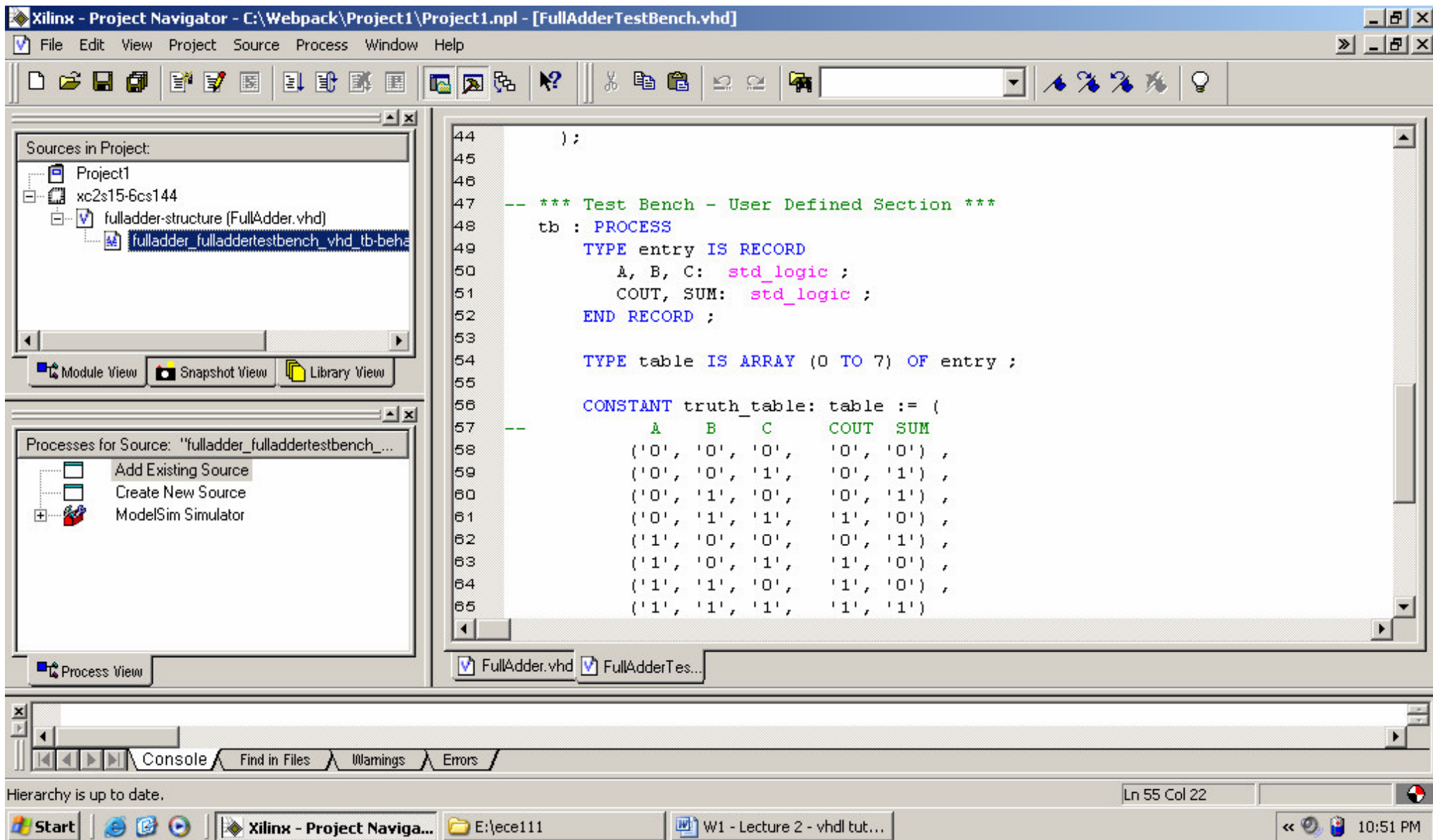
If you have a list of modules already created, you will see it here. Click “Next” to continue.



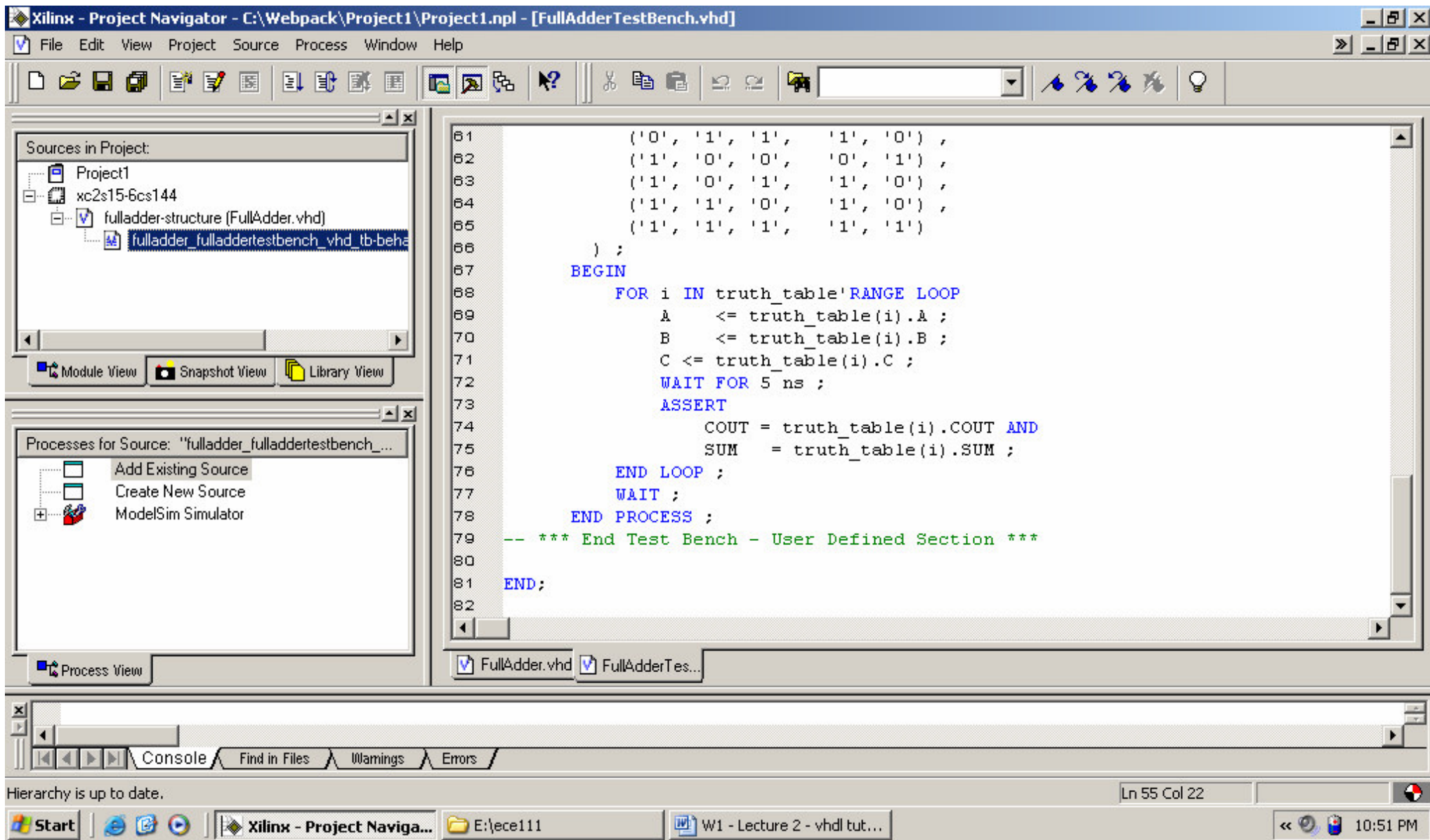
Click "Finish" after you verify what you about to create is correct.



A test bench template will be created, similar to VHDL module but with different library & declarations.



Add in the test bench code. This example shows a complete combination.



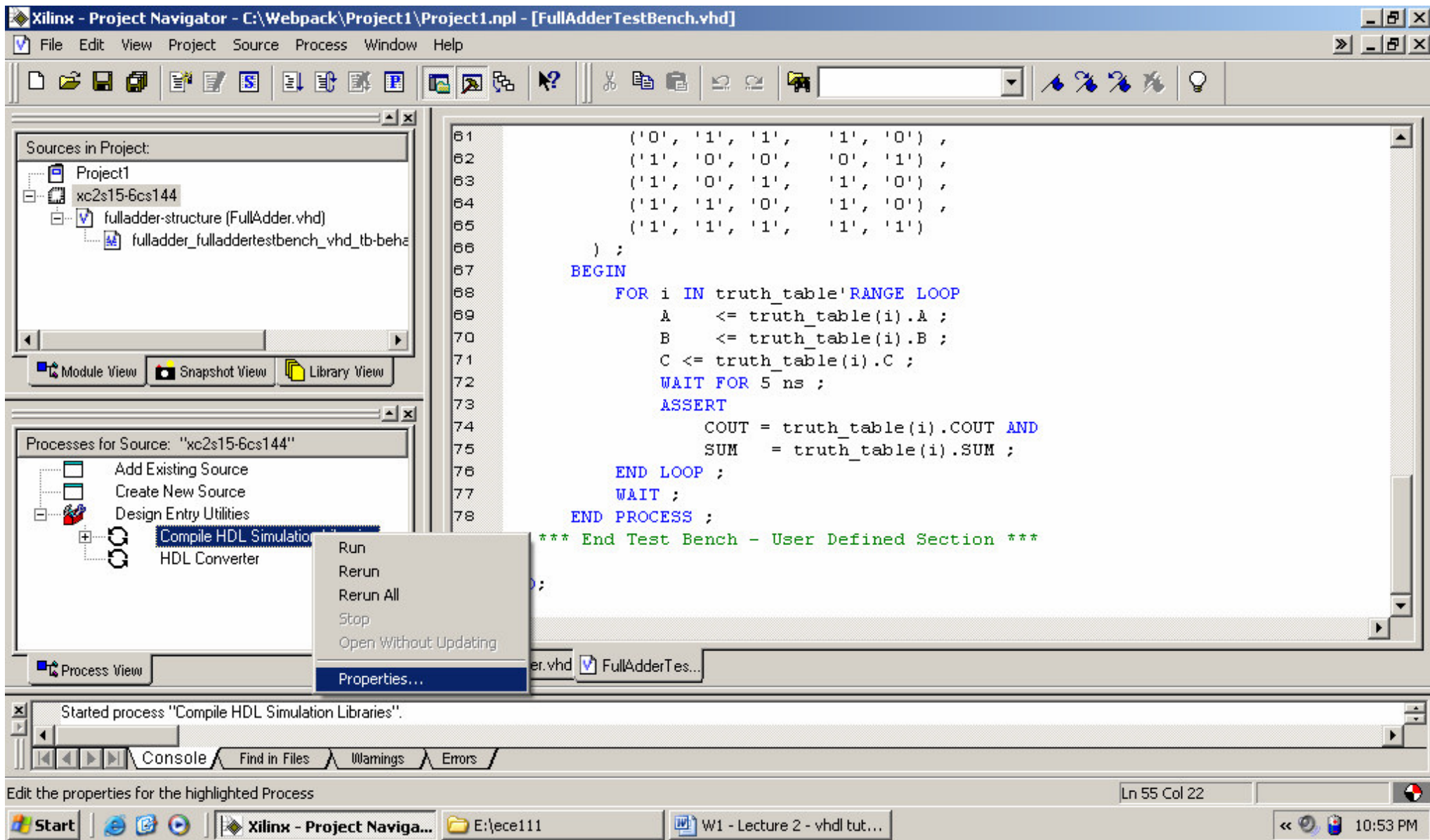
The ASSERT will provide an error in the main window (not the waveform window) if value COUNT & SUM values do not map to the truth table.

Another example of using assert statement:

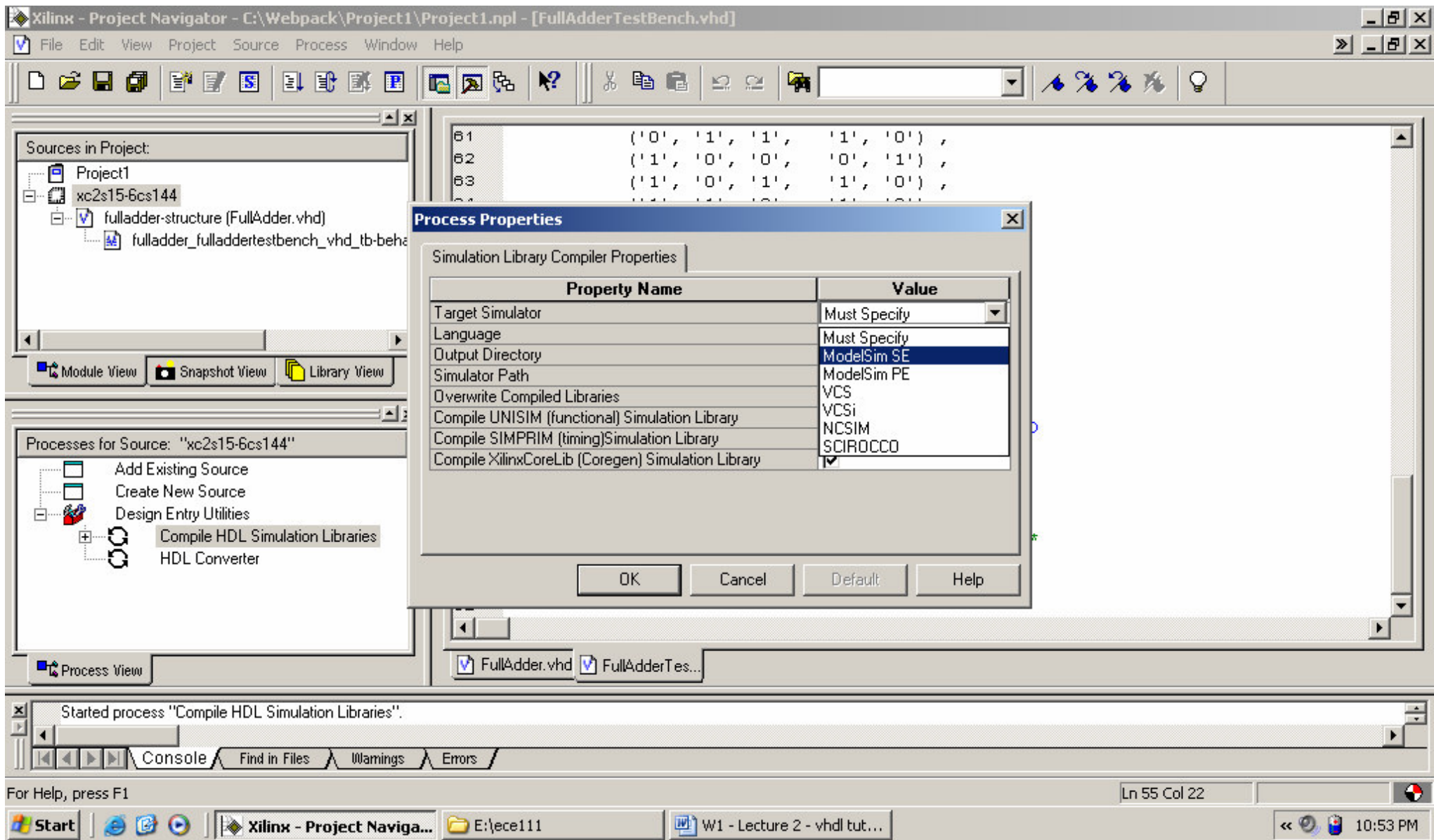
```

assert COUNT = '0'
    report "COUNT output incorrect. Value 1 is expected instead"
    severity error;

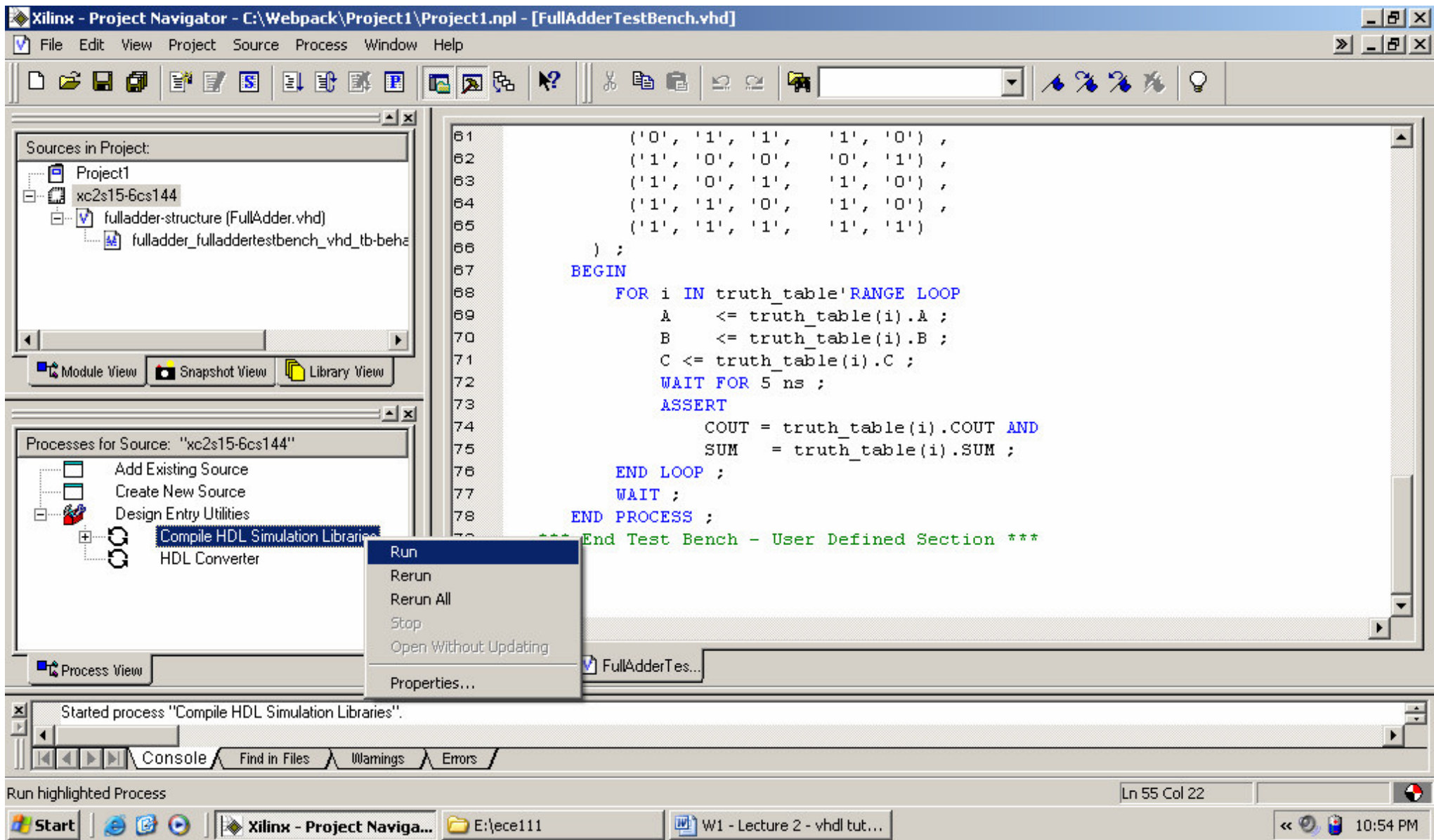
```



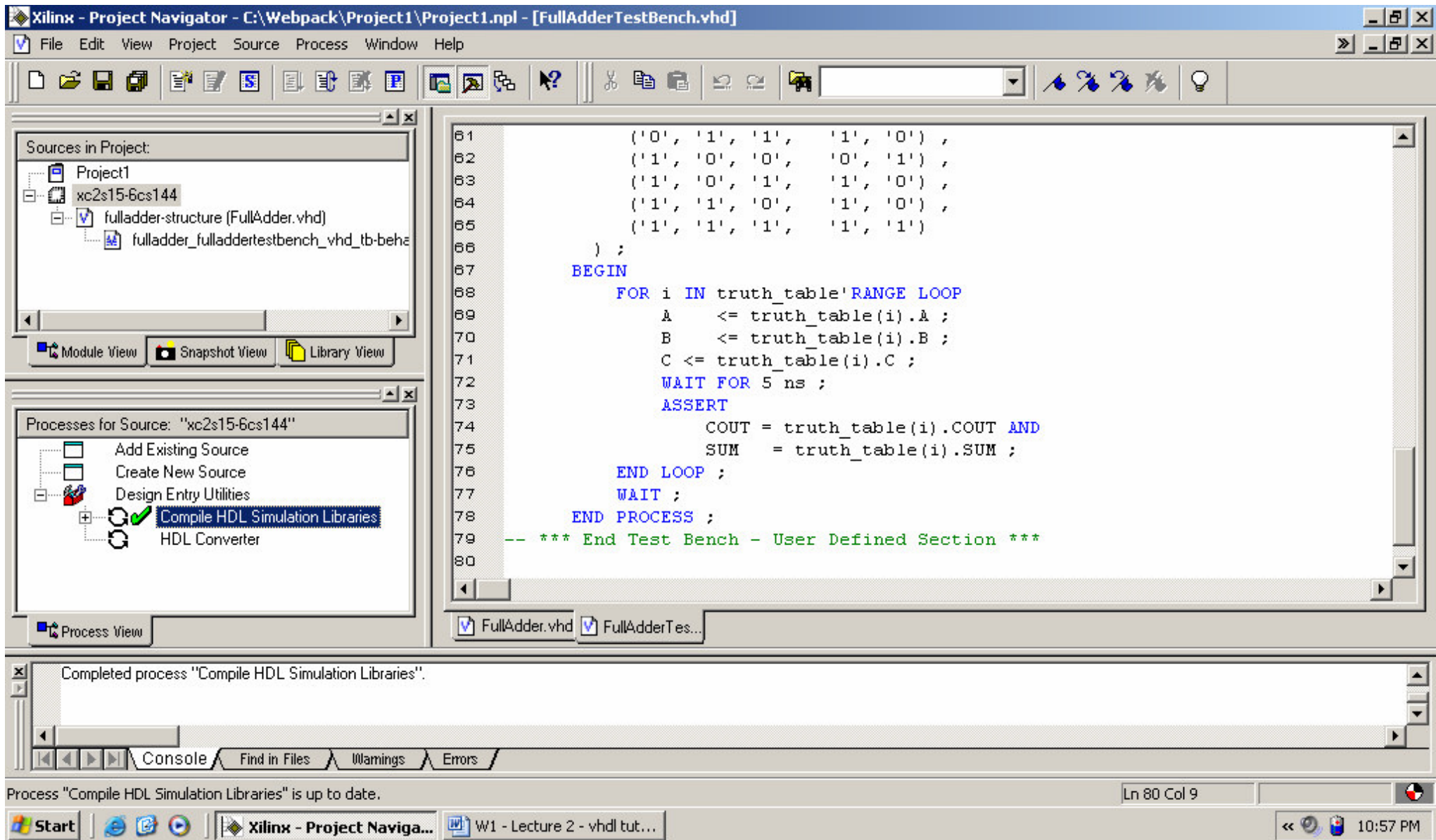
Before you can simulate a VHDL module, you need to tell it which simulation you are going to use. Make sure you have the xc2s15-6cs144 selected on the top left window, then right click on the Compile HDL Simulation > Properties...



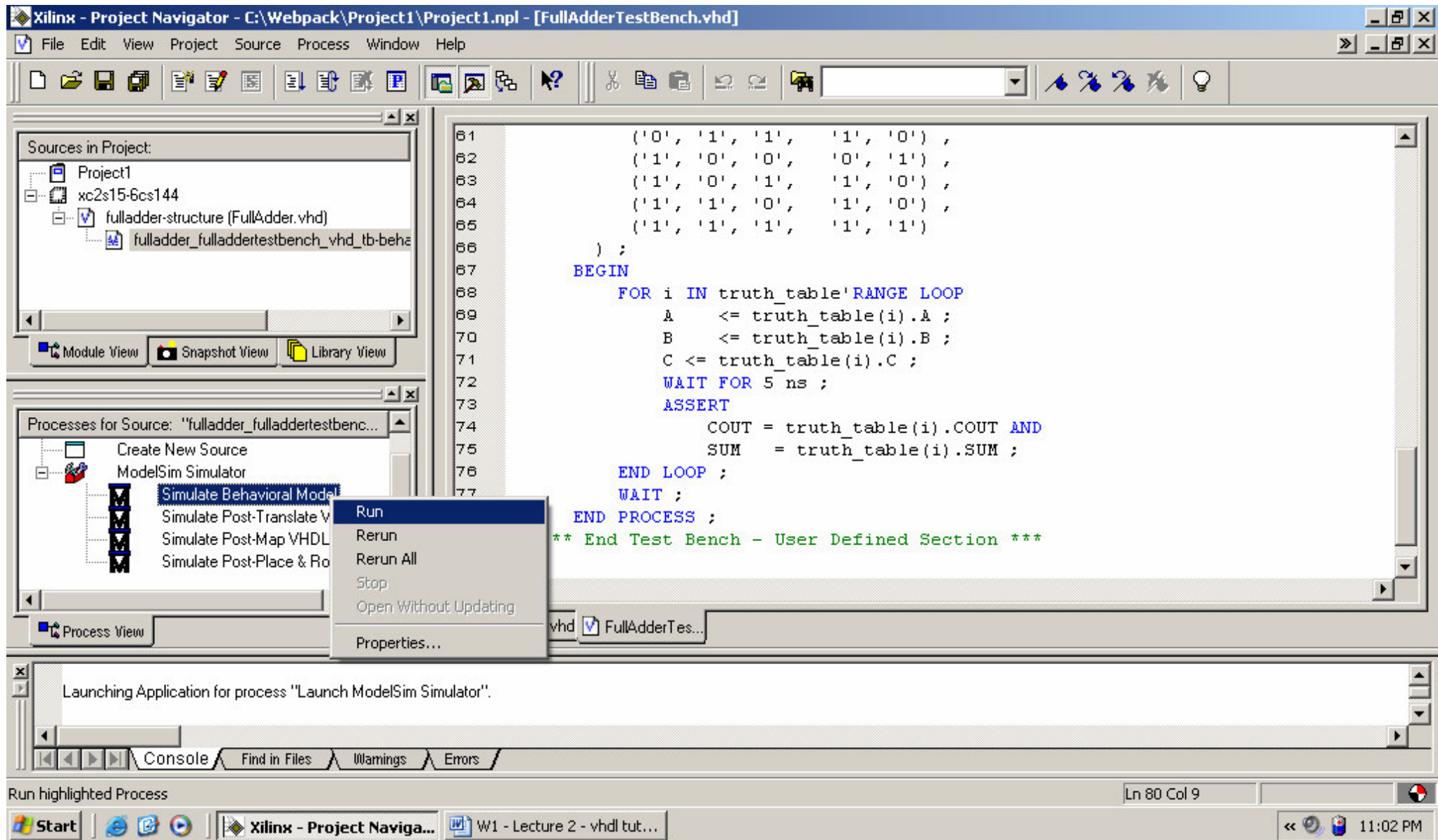
Select ModelSim SE which is the freeware version of simulation you have installed before.



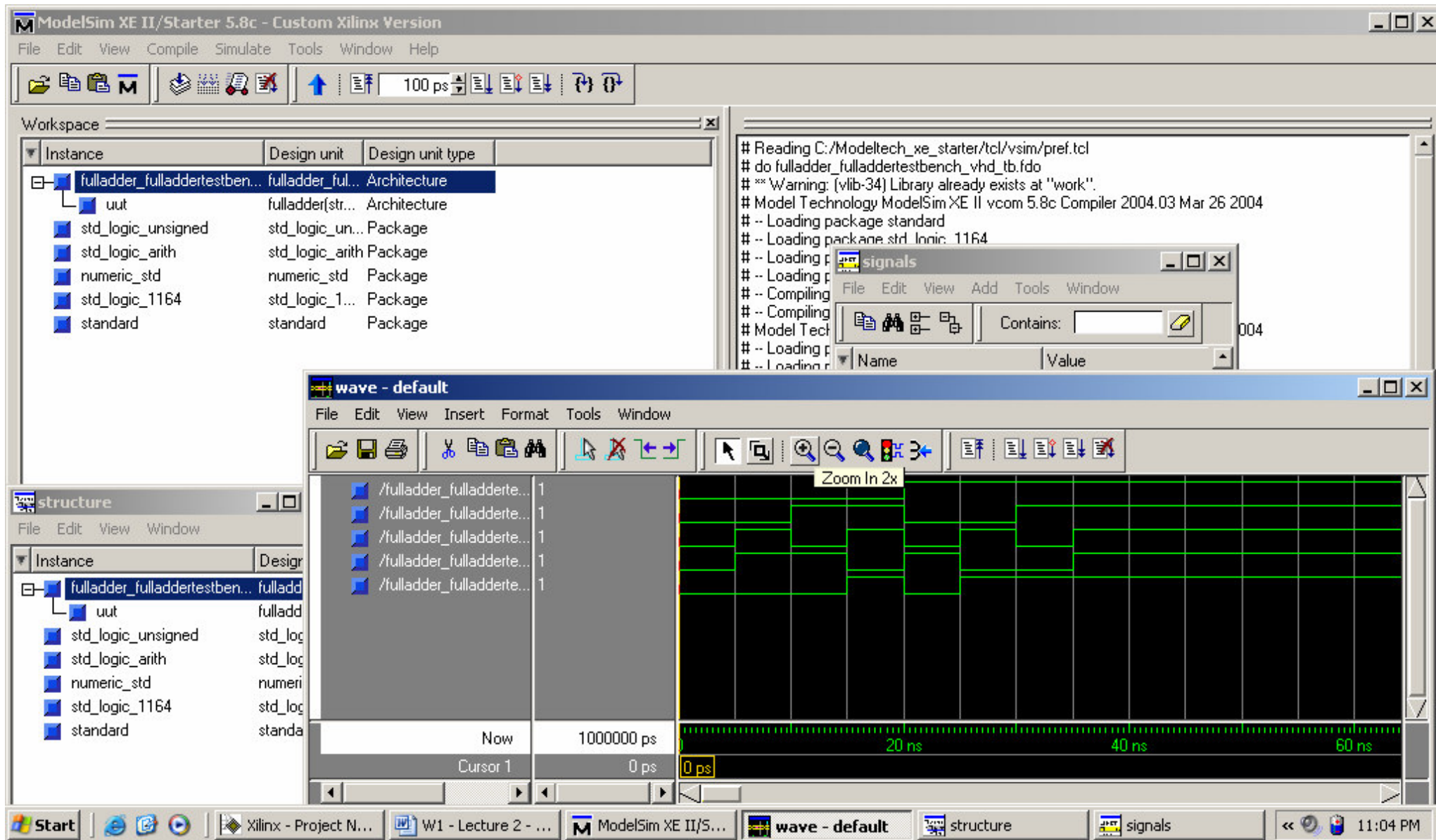
Now you are ready to compile the simulation library so you can run the simulation later.



When it's finished compiling the libraries, you see the Green Check Mark next to it.



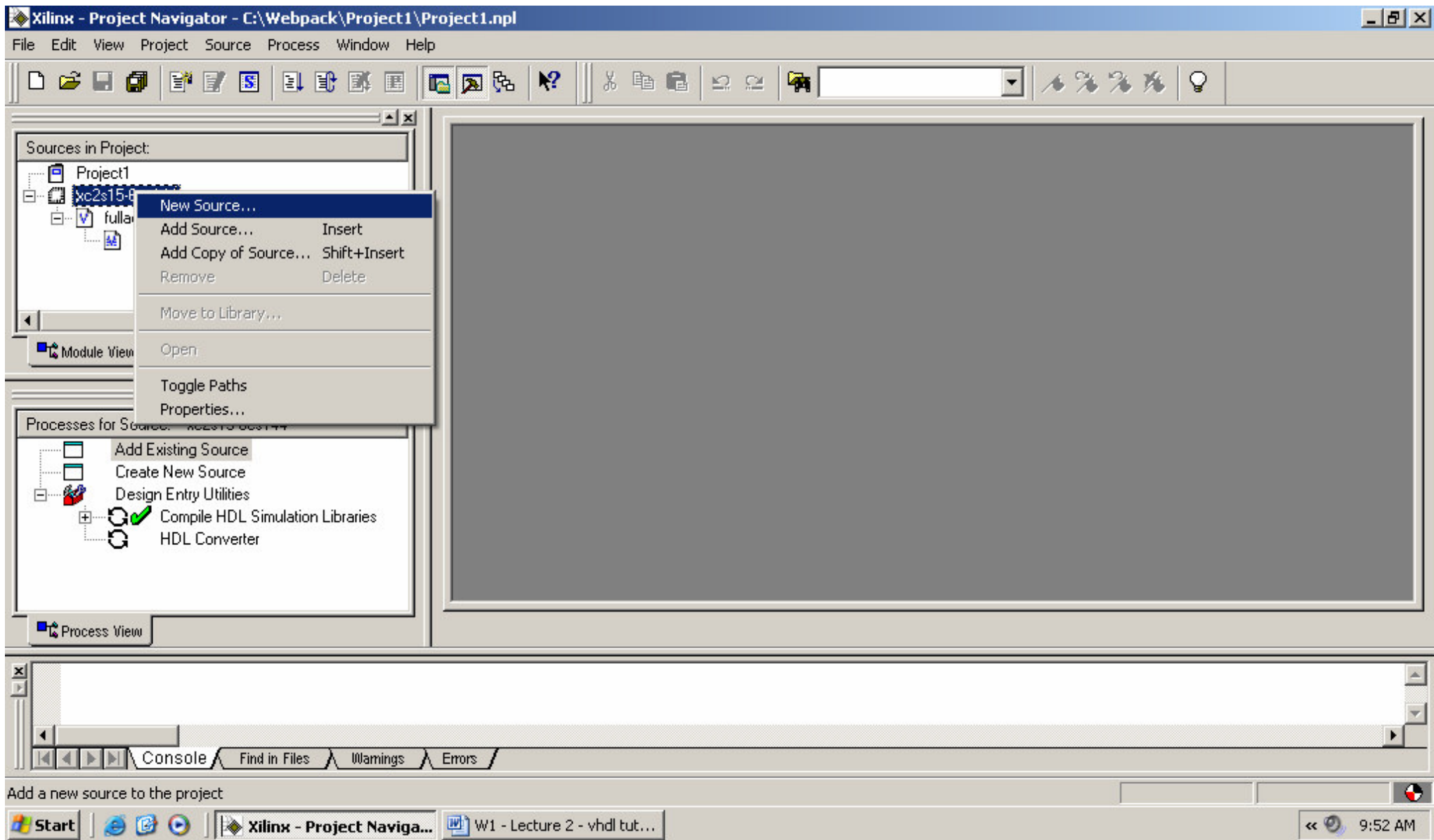
Now, select the test-bench file on the upper left window. Then the bottom window will show the ModelSim simulator option. Right click on the Simulate Behavioral Model to run the simulation. It will open up few windows.



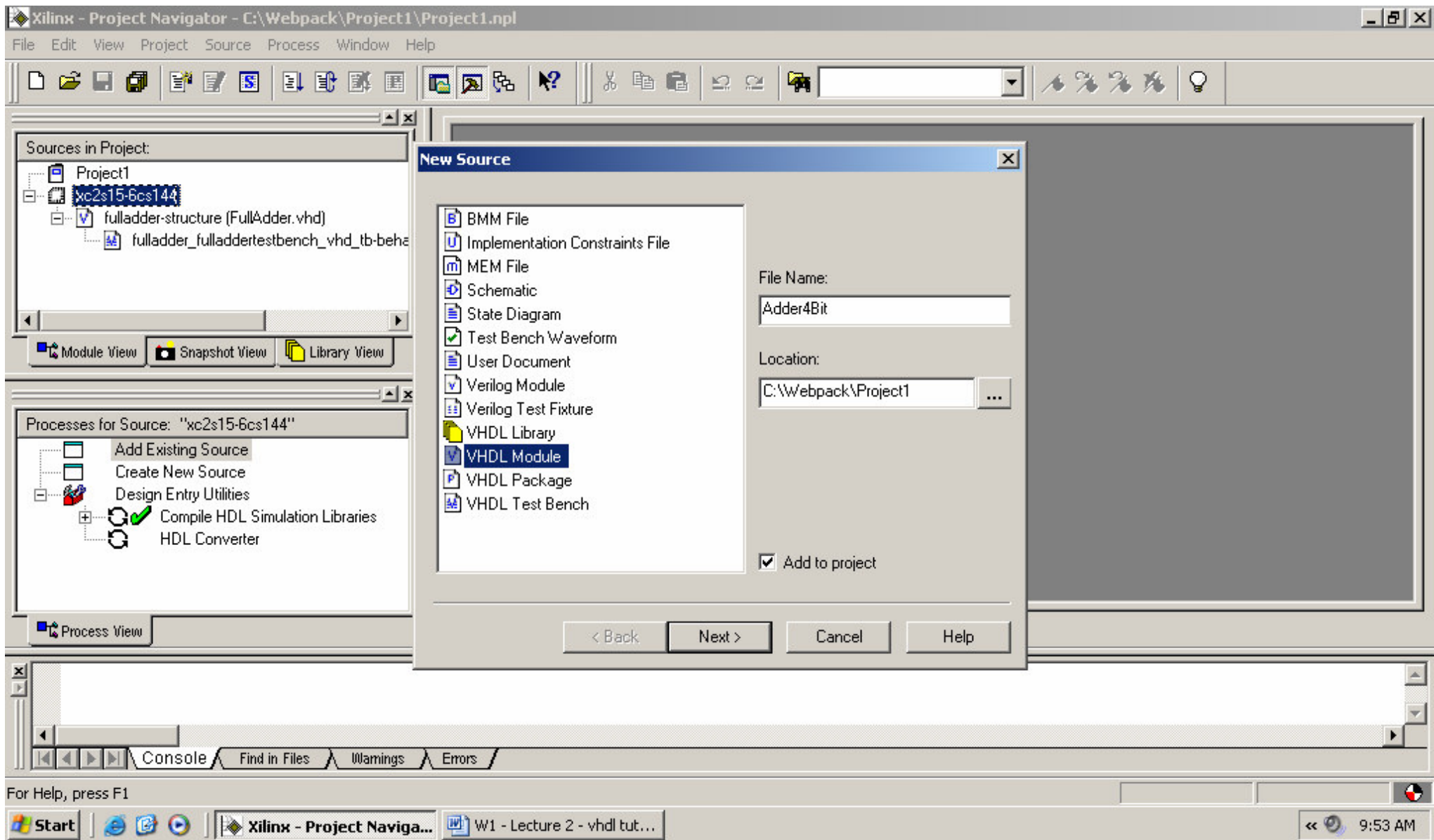
Note, you may need to experiment with the zoom in/out and horizontal scroll-bar in the wave-form!

Note the window behind the wave form with a list of status (# Reading C:/Modeltech_xe_starter/tcl/vsim/pref.tcl), if an error occurred with the simulation, error messages will appear in that window.

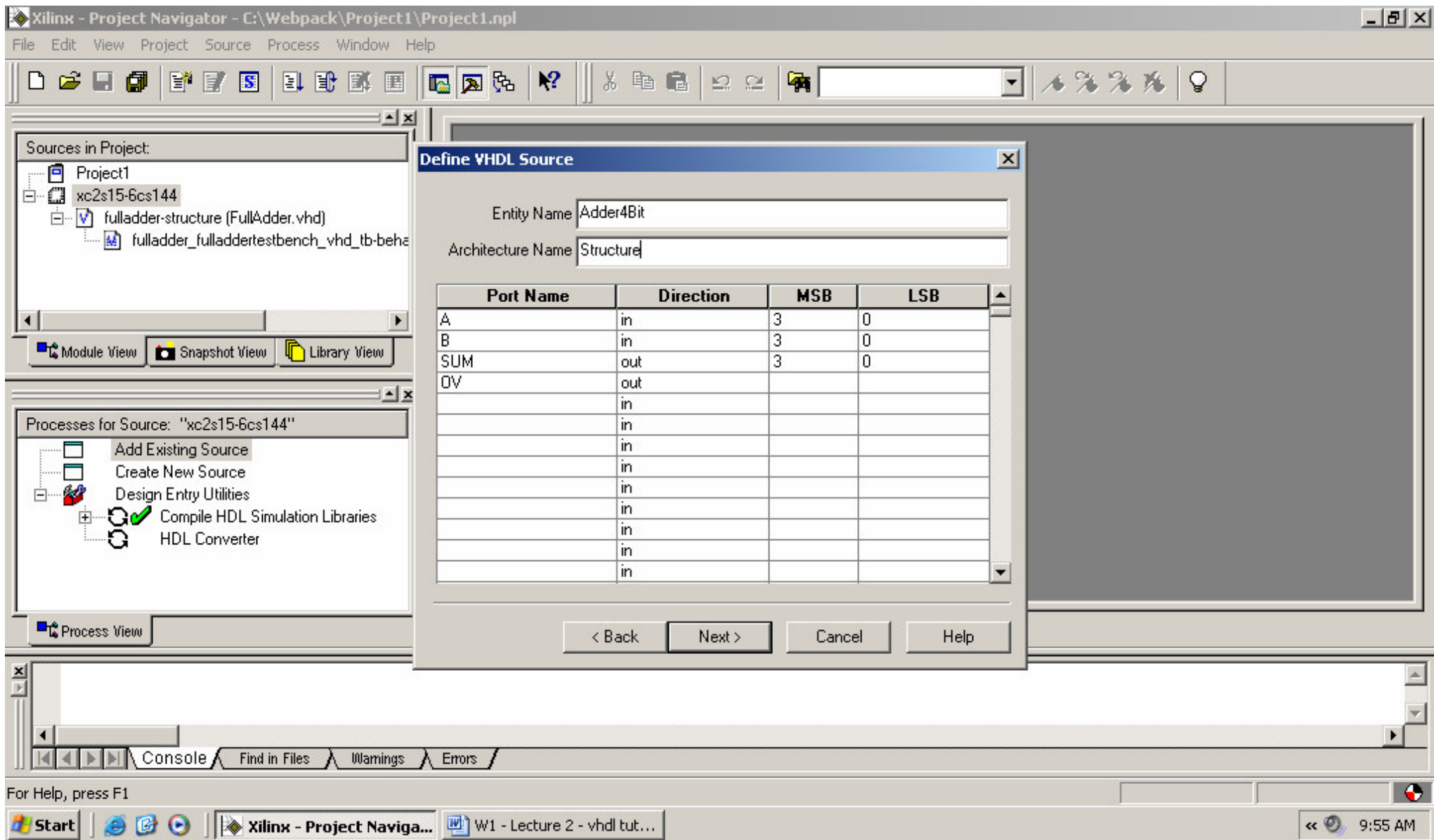
(After you verify the simulation is correct, you need to close the ModelSim. Otherwise, it will try to launch another one next time)



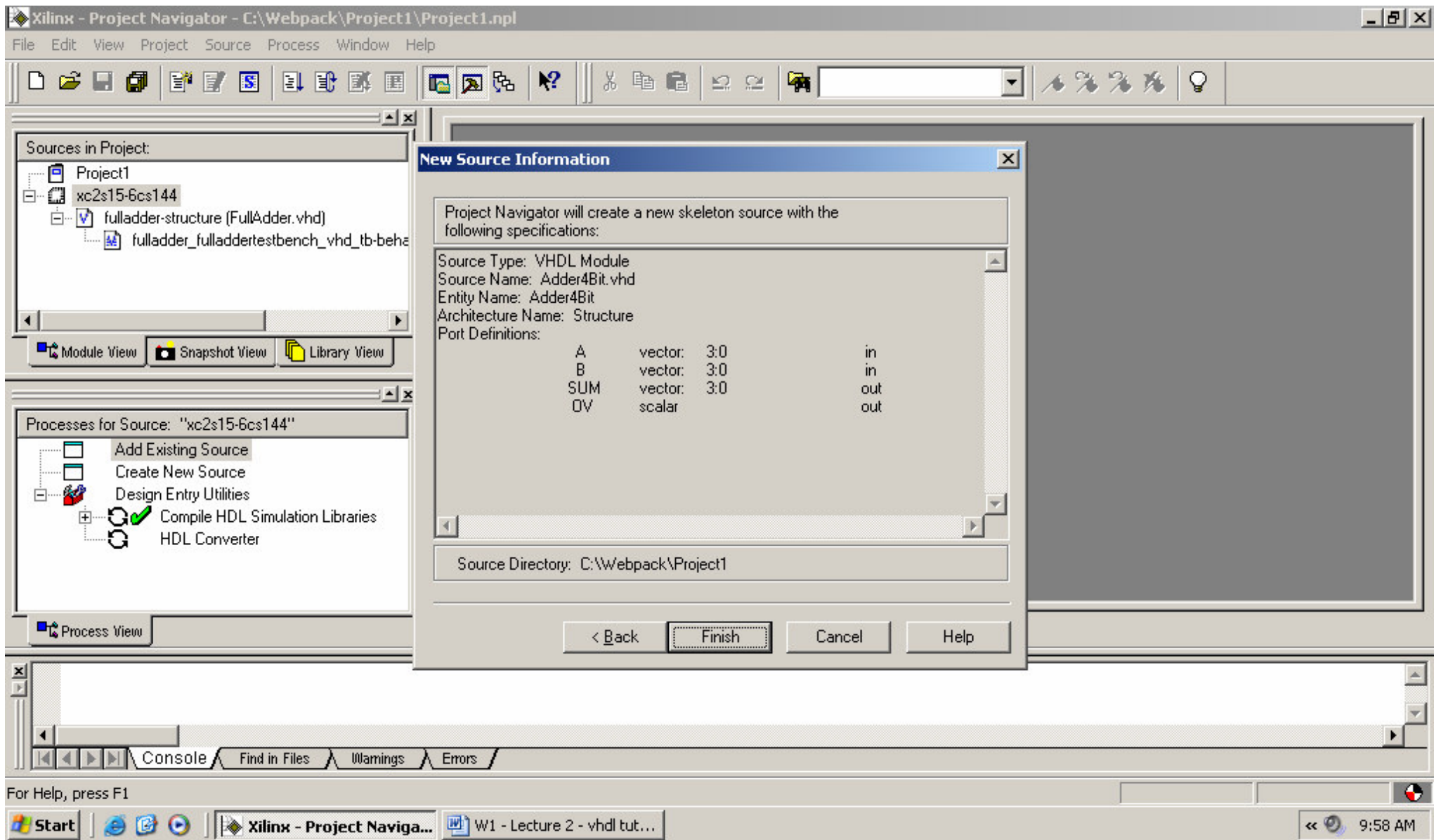
Let's create a 4-bit adder based on the full-adder we just built.



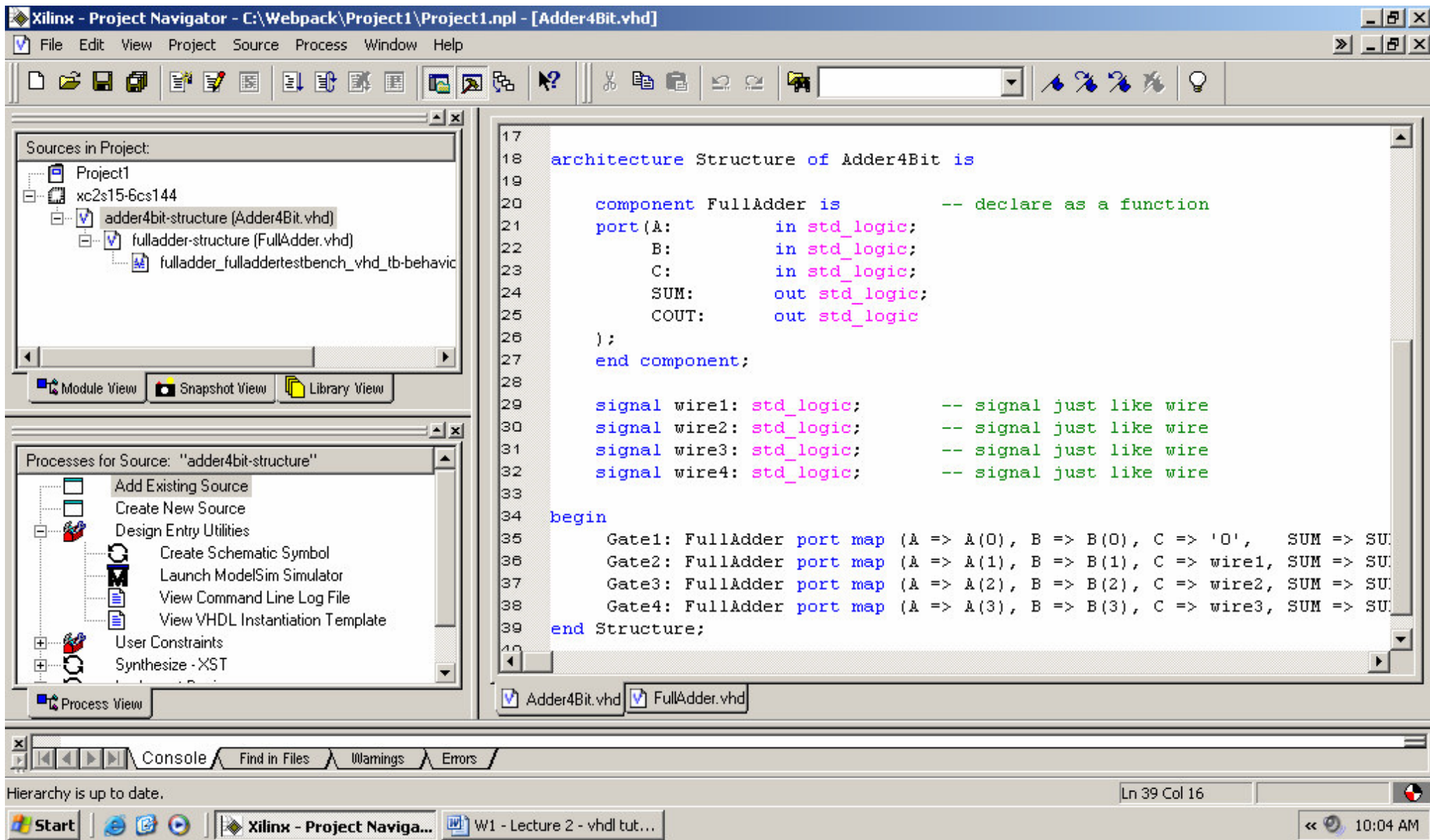
Same as before...



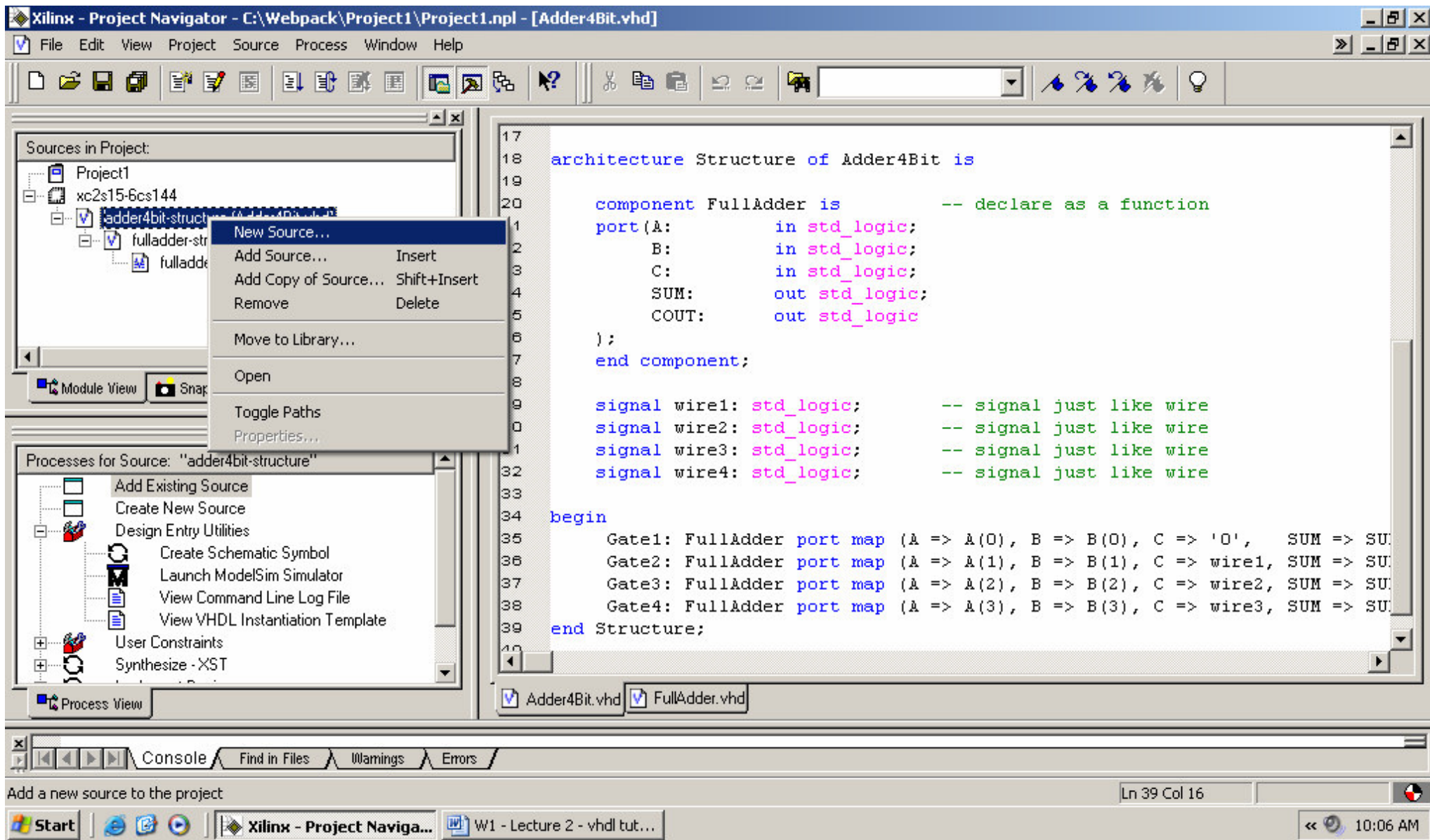
Note, we are setting A(3:0) and B(3:0) as input and SUM(3:0) as output by assigning MSB values
It is also a “Structure” architecture



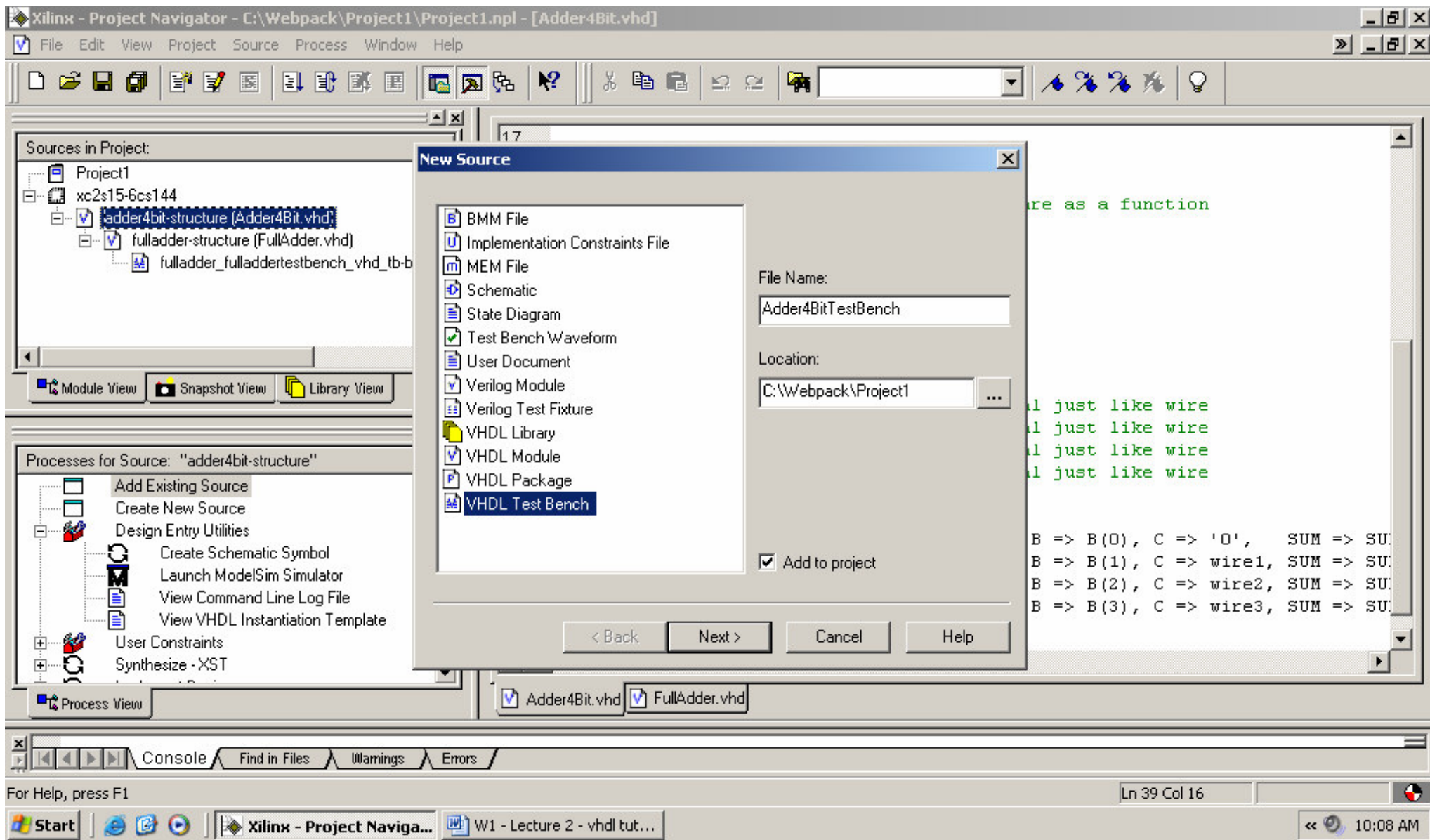
Looks good! Click "Finish".



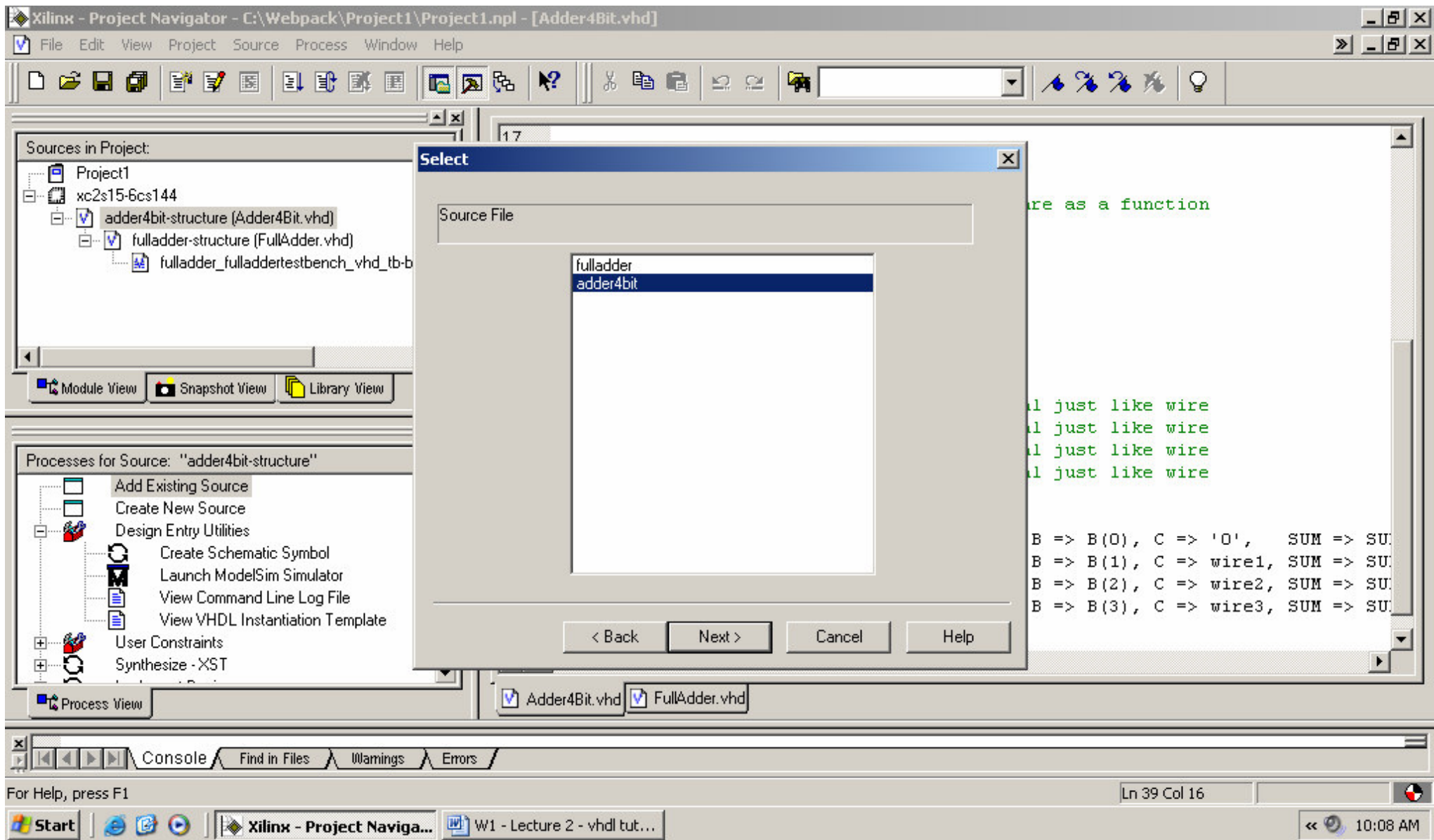
Here is the code for 4-bit adder by mapping 4 FullAdder components within the “begin” and “End structure”. It’s a ripple adder (carry-out of first full-adder goes to the carry-in of the next full-adder)



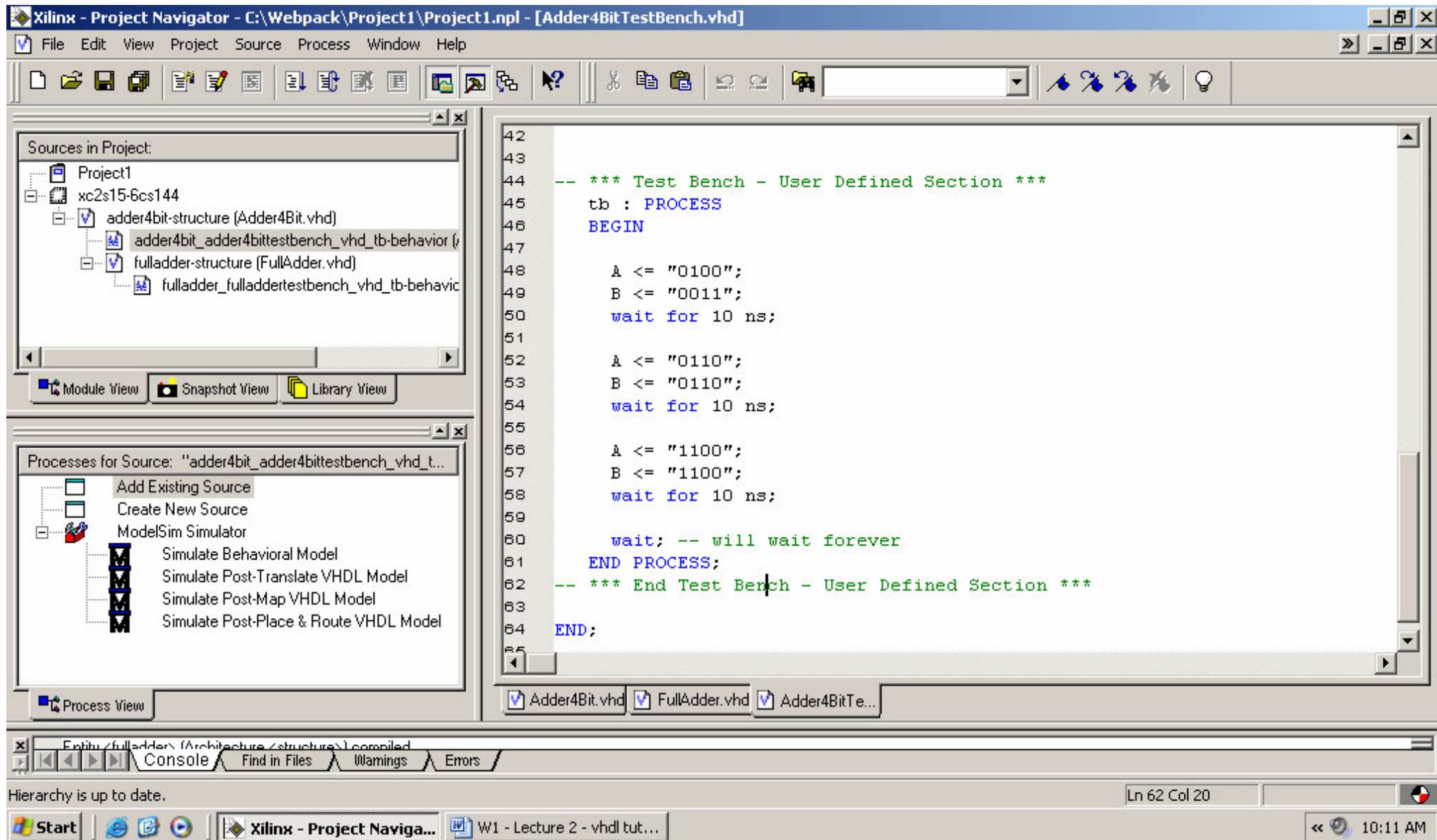
Now, add a simple test bench for 4-bit adder.



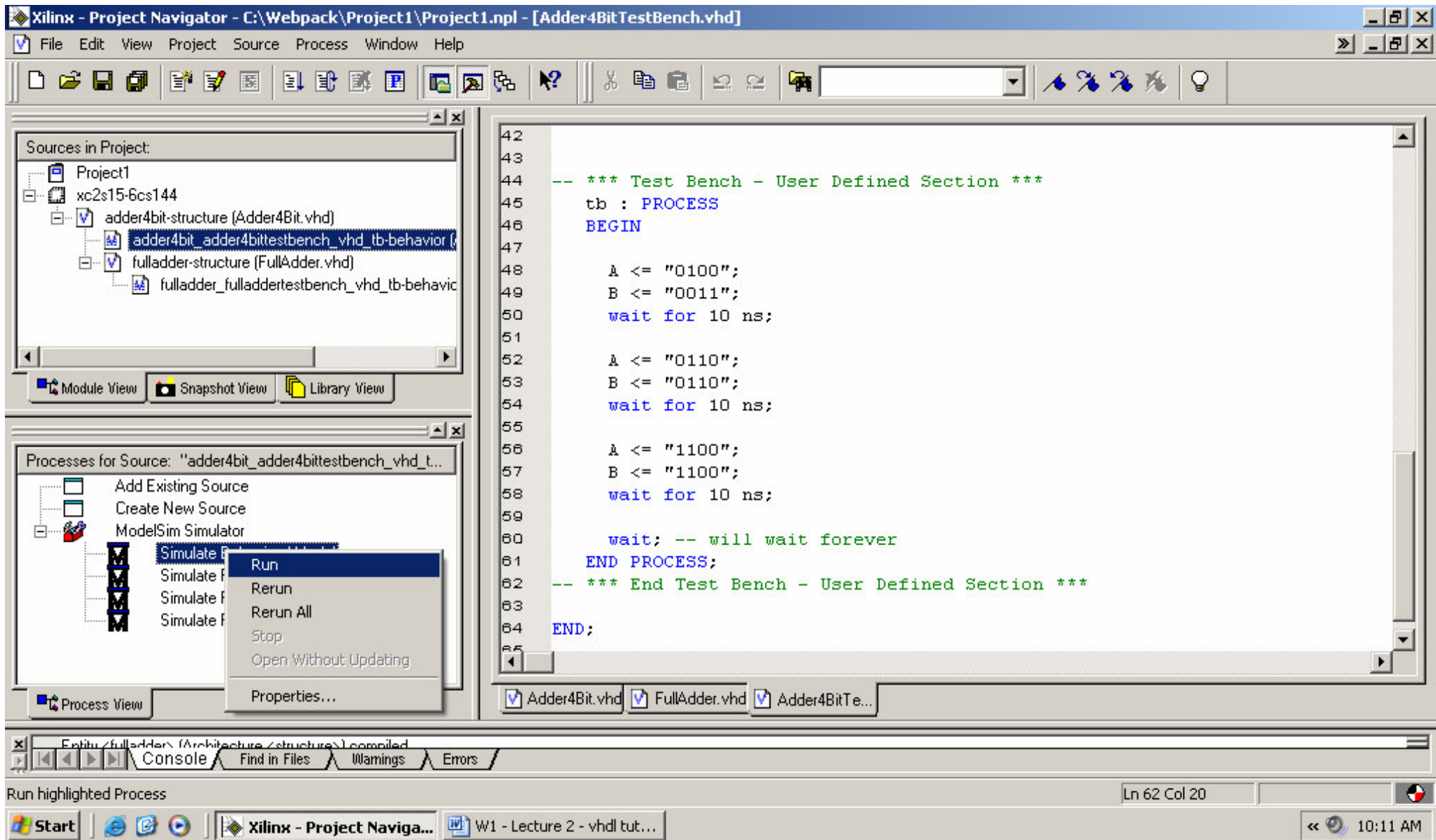
Same as before...



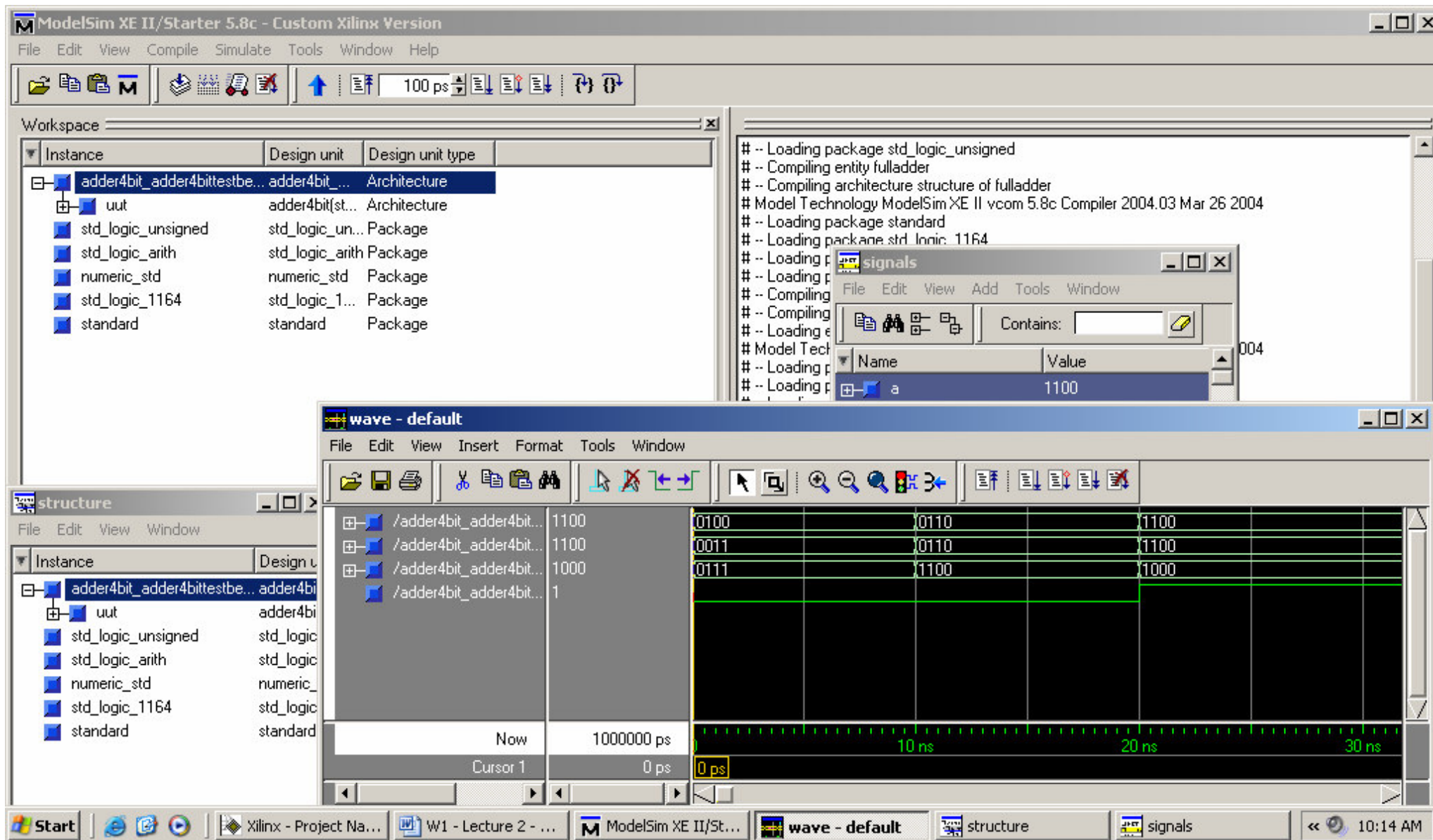
Make sure adder4bit is selected. Click “Next” and then Finish on the next screen (omitted)



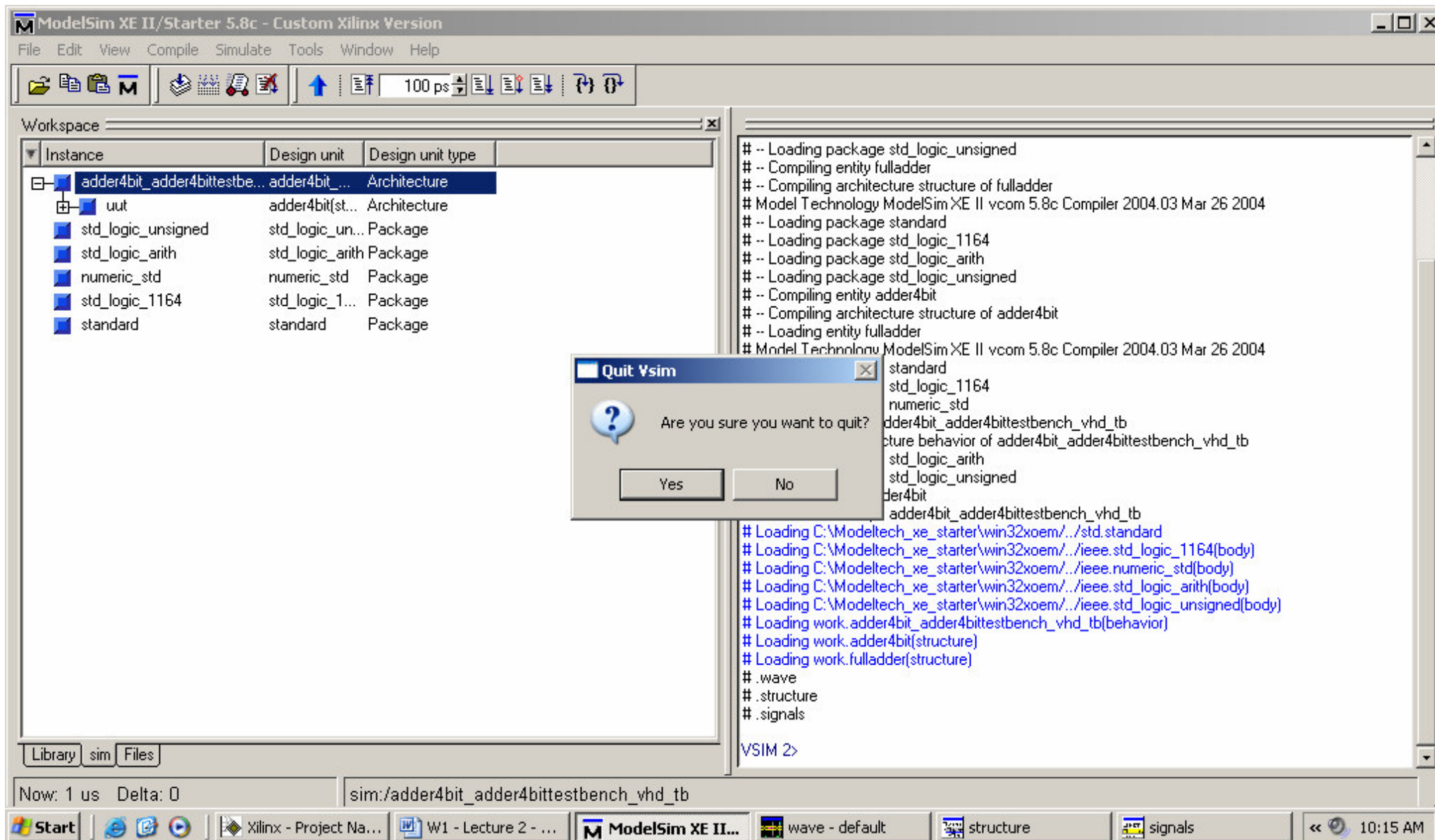
Add the simple test bench code, only three cases we tested in this case. Sometimes, you just can't test every possible case.



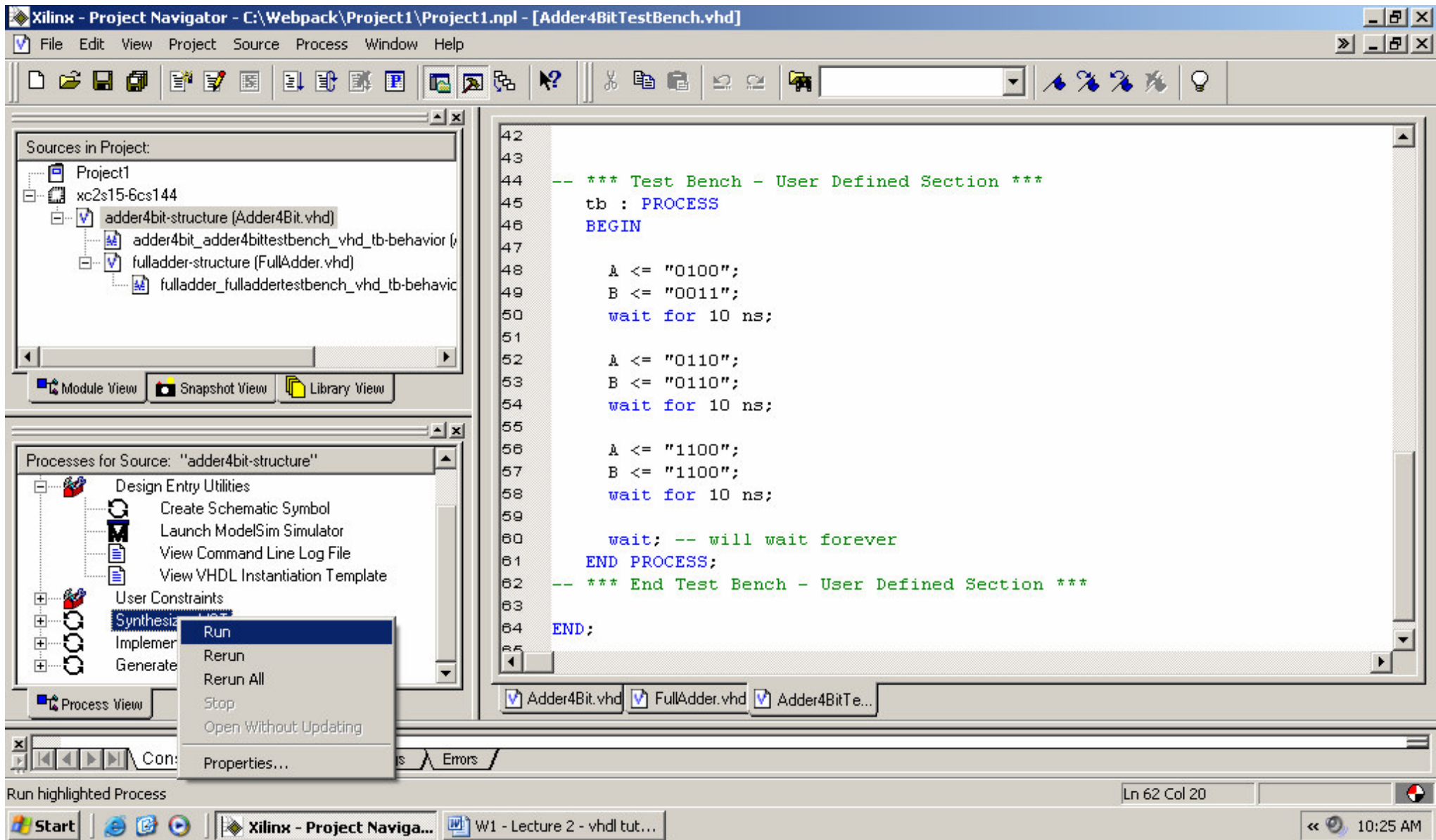
With the test bench file selected on upper-left window, right click on the bottom left window's "Simulate Behavioral Model" and Run.



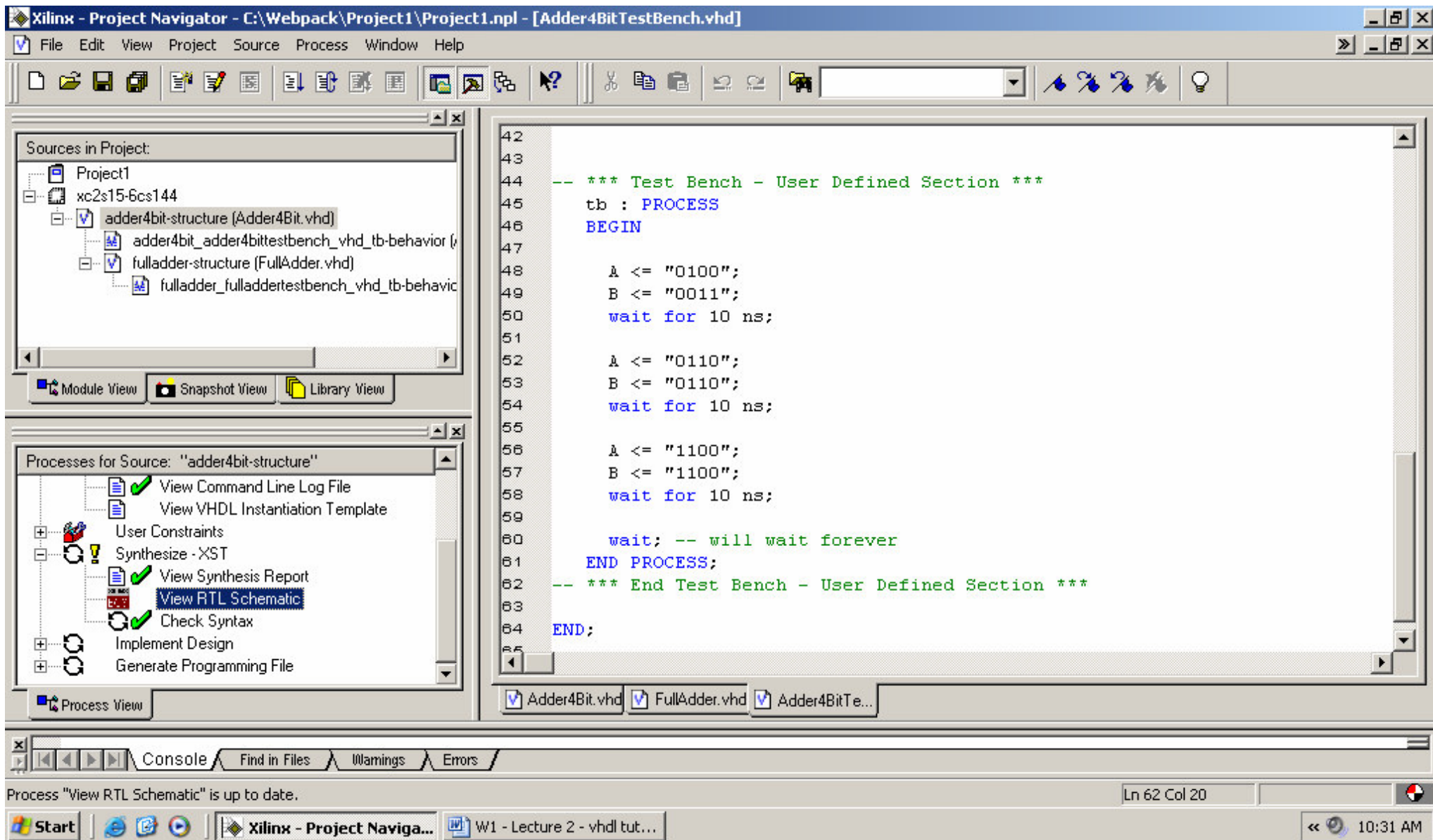
Note, you have to zoom in/out a bit and scroll the waveform window to the left.



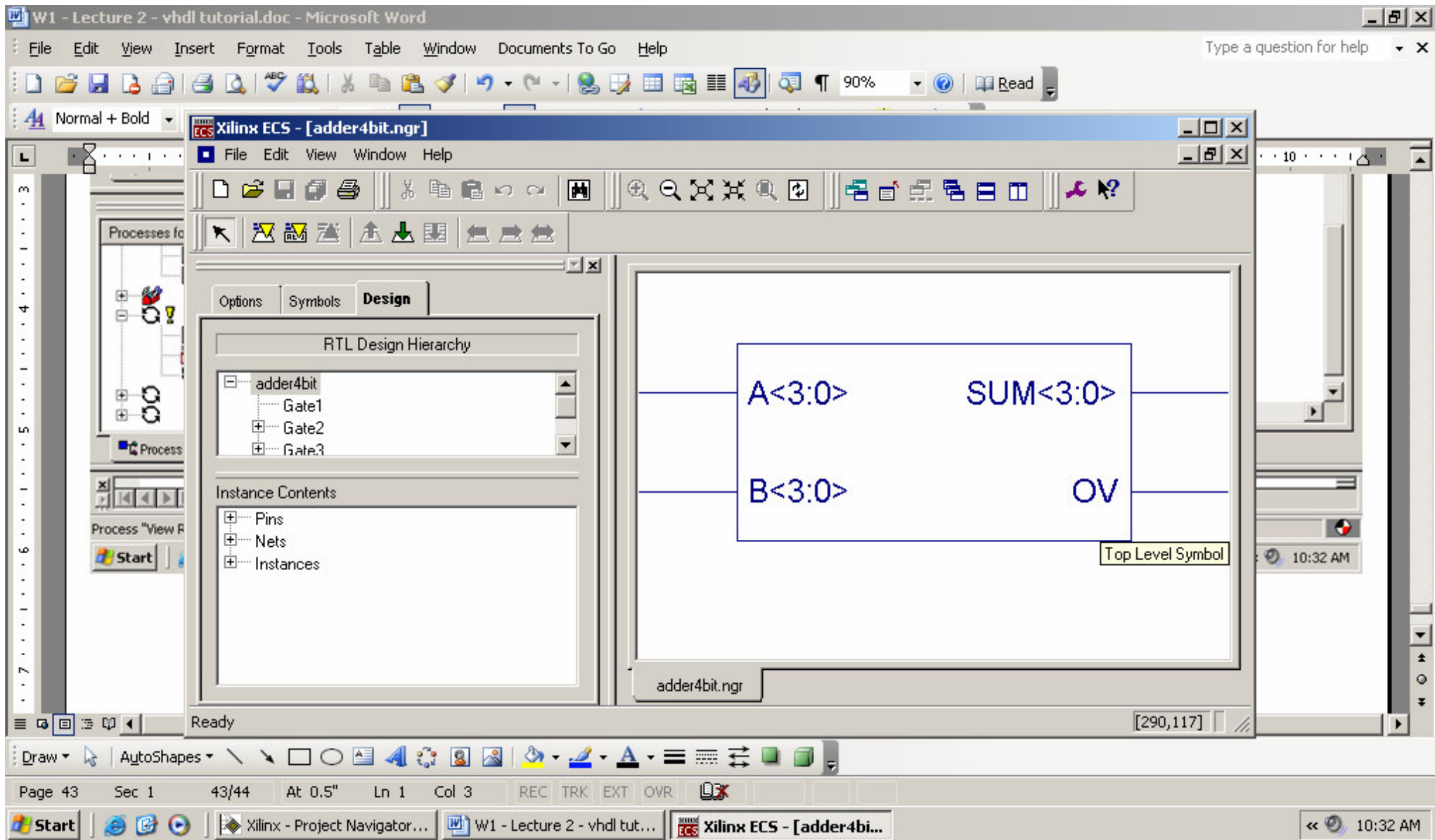
Again, make sure you exit ModelSim each time.



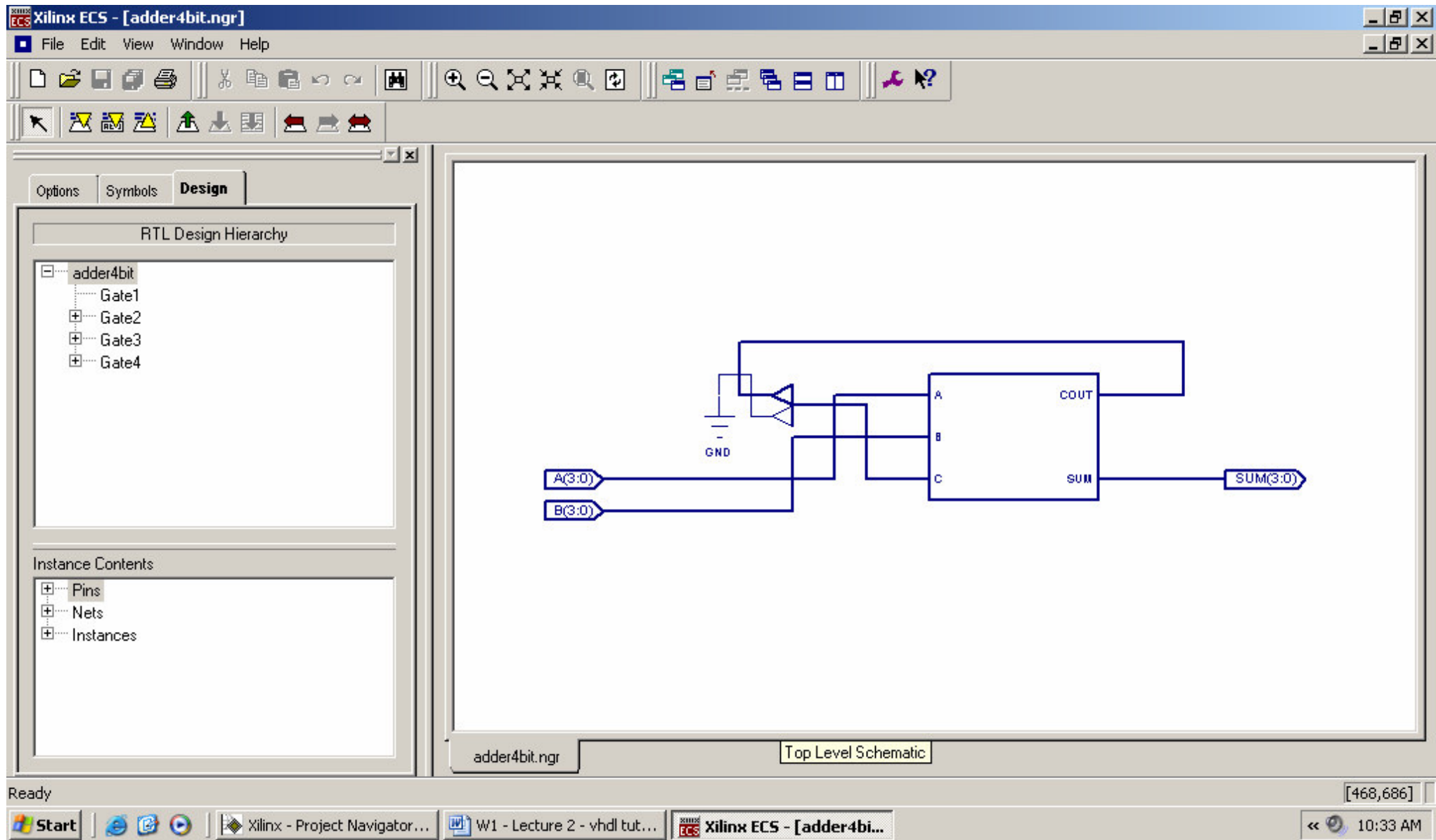
With 4 bit adder selected, click on the Synthesis (generating the actual logic circuit).



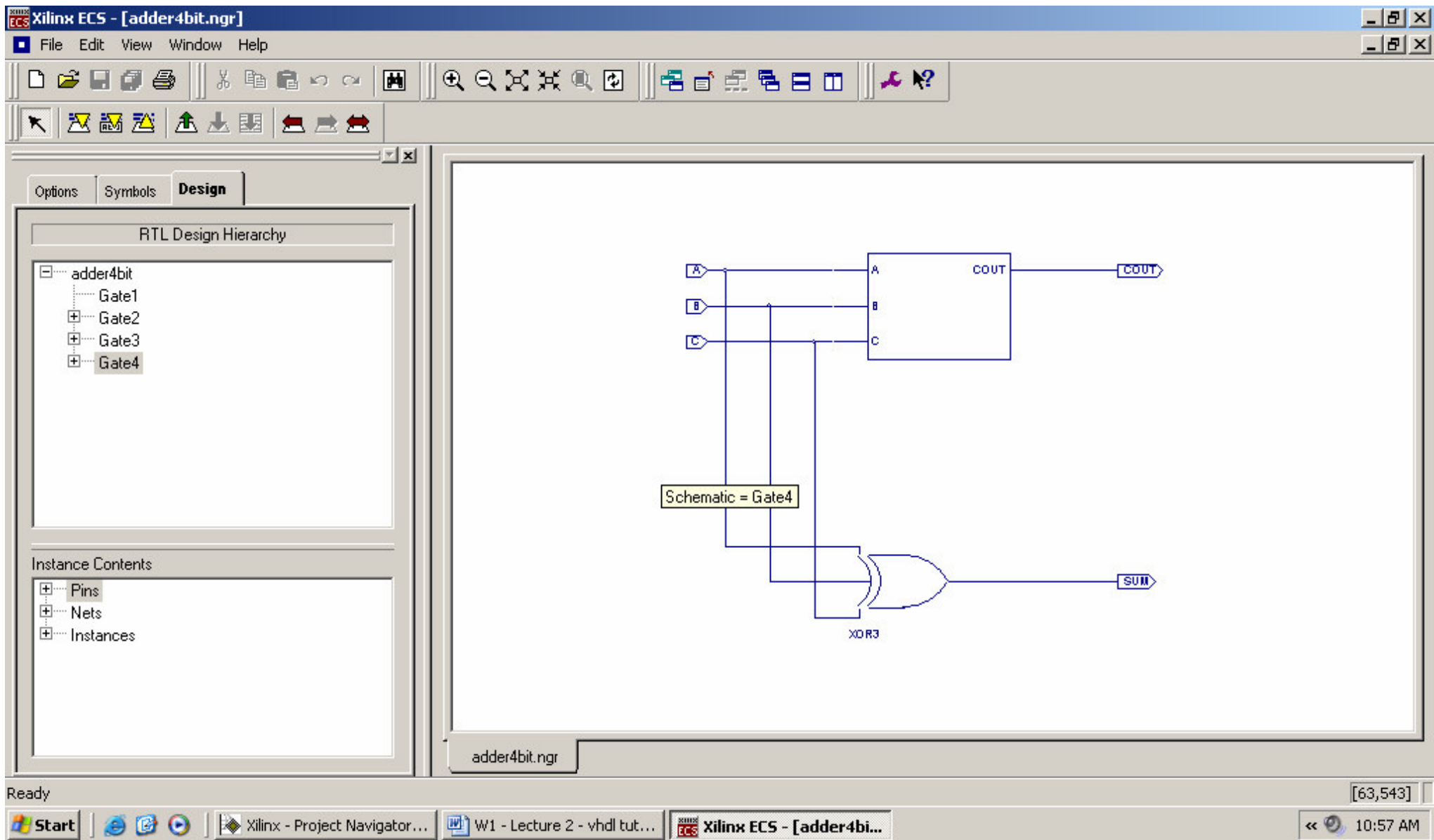
Expand the synthesise-XST dir on the bottom-left window, and double click on “View RTL Schematic”



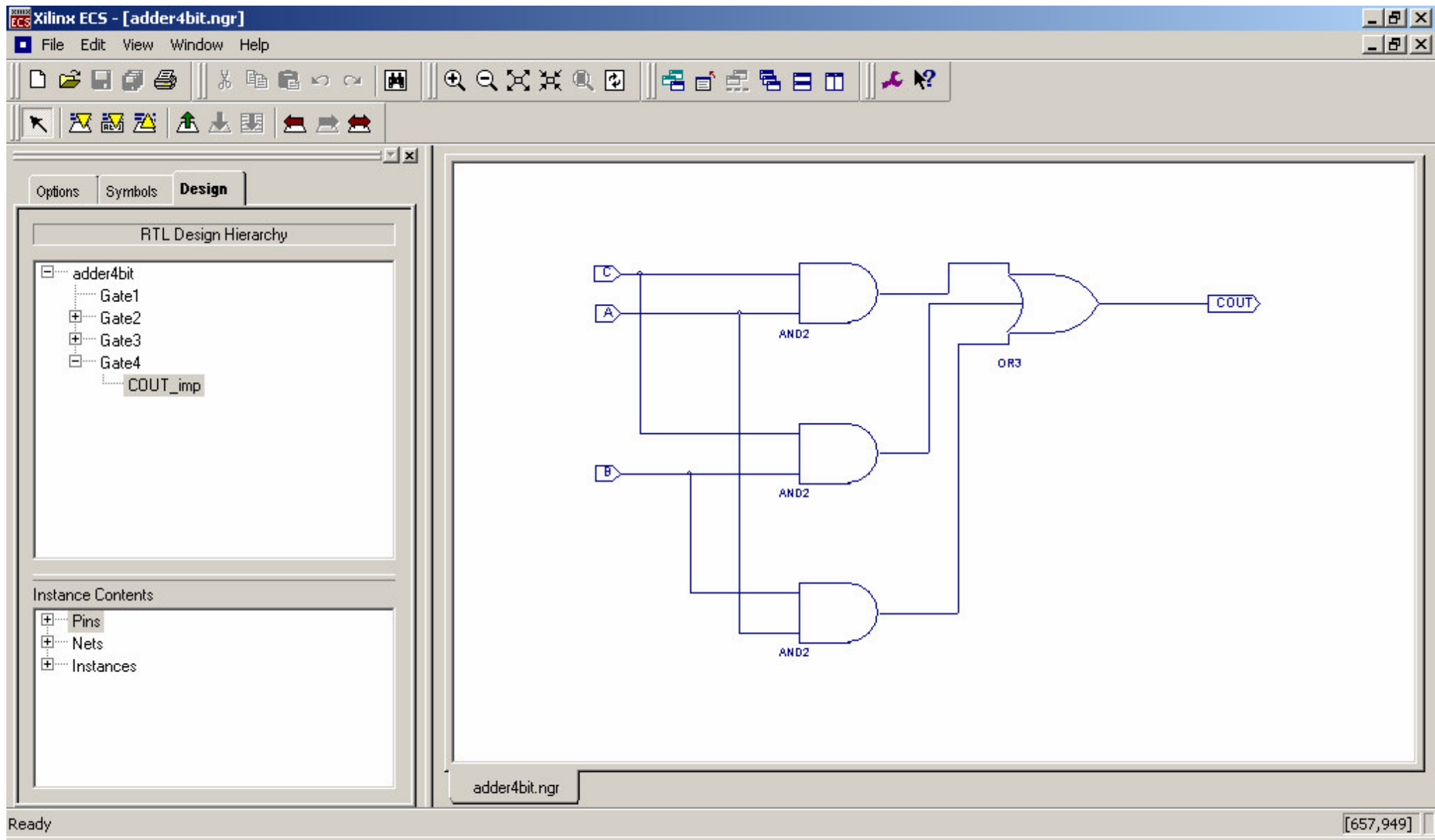
You can drill down what is inside the 4-bit adder by double click on the schematic symbol



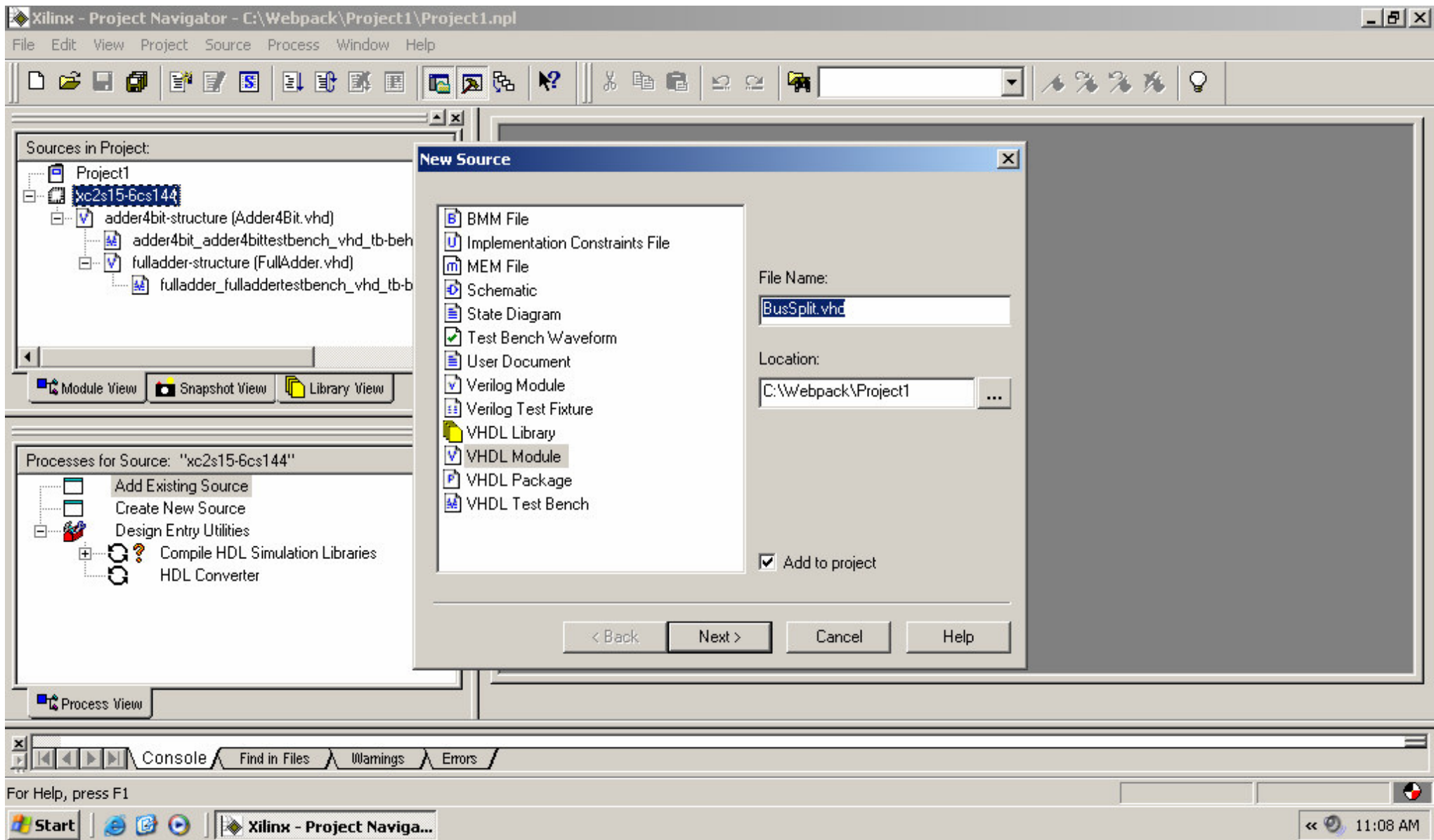
Double click again at the schematic and you'll see the full-adder implementation



Double click again on the component

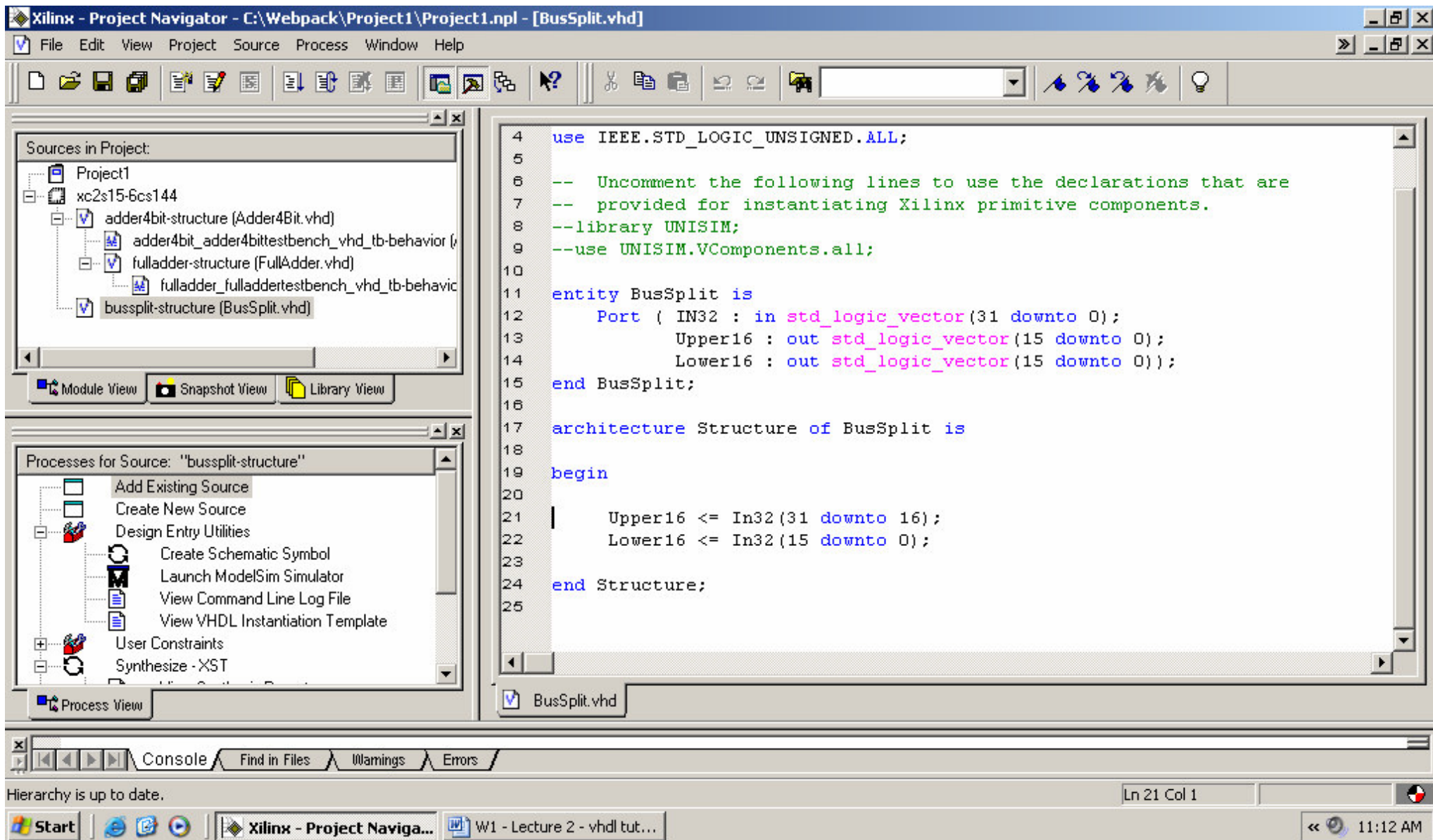


This is as far as it goes. Close this window.

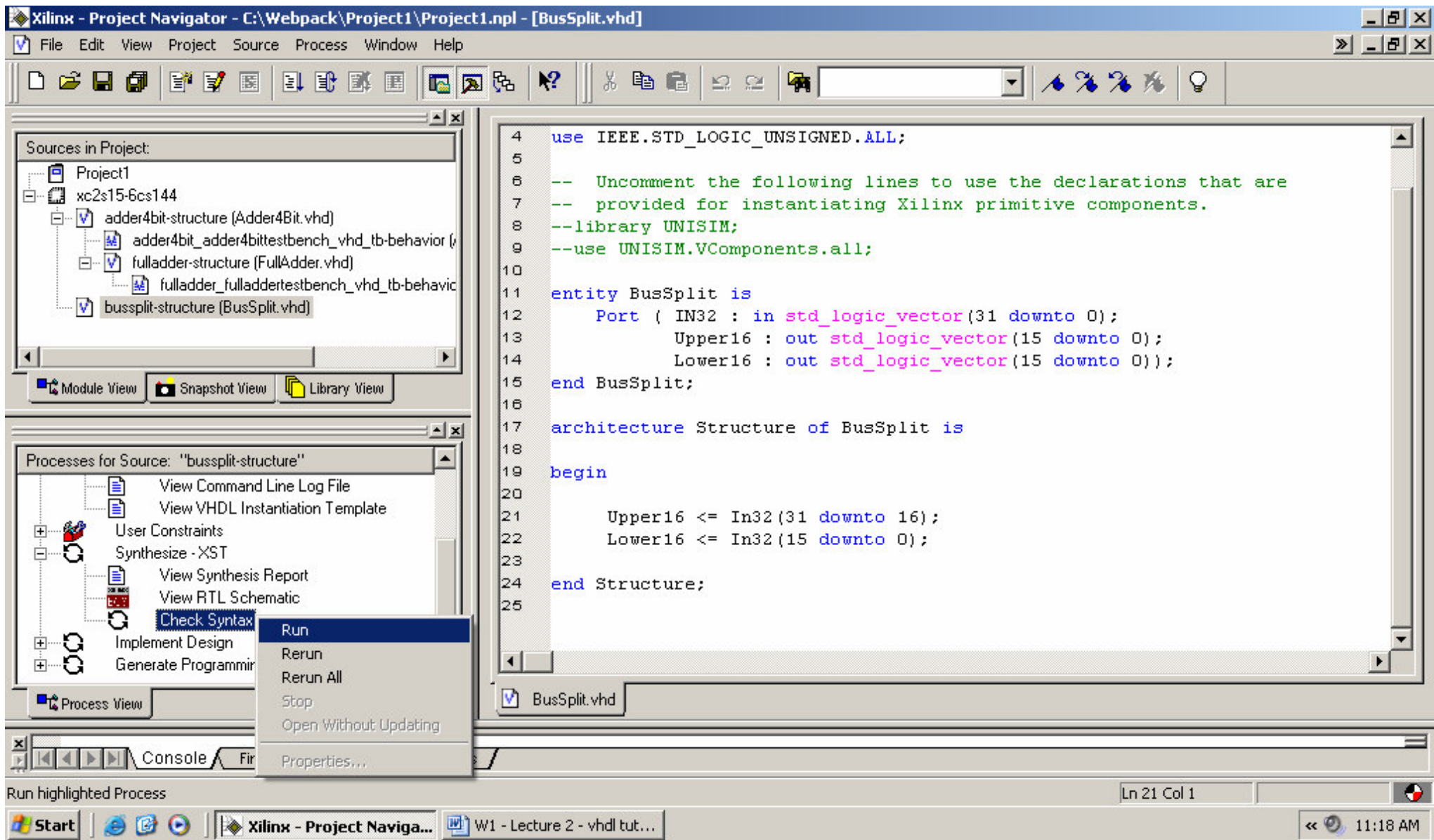


This is another example.

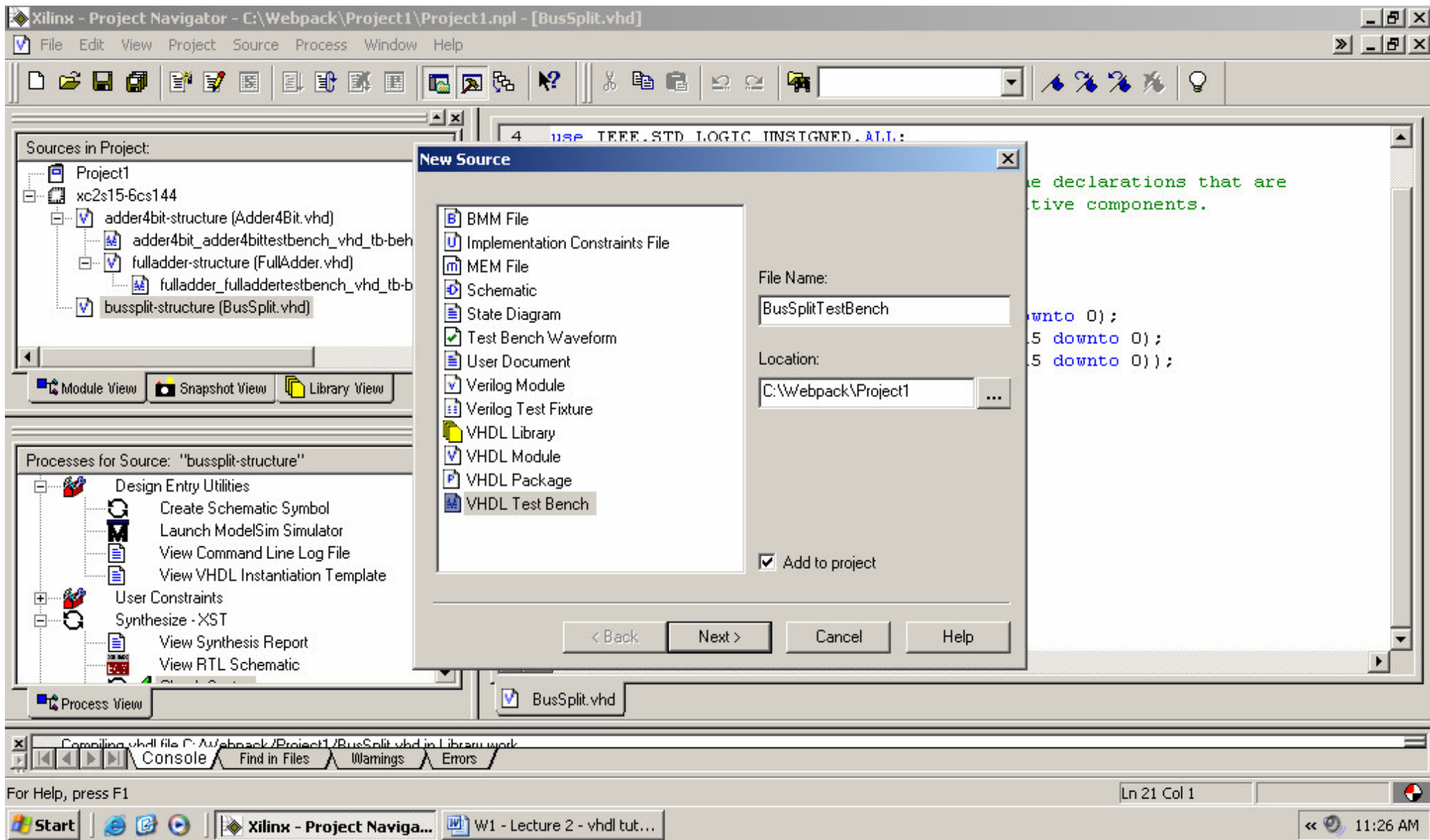
A bus-splitting module. Splitting 32-bit input into two 16-bit output (upper 16 & lower 16)



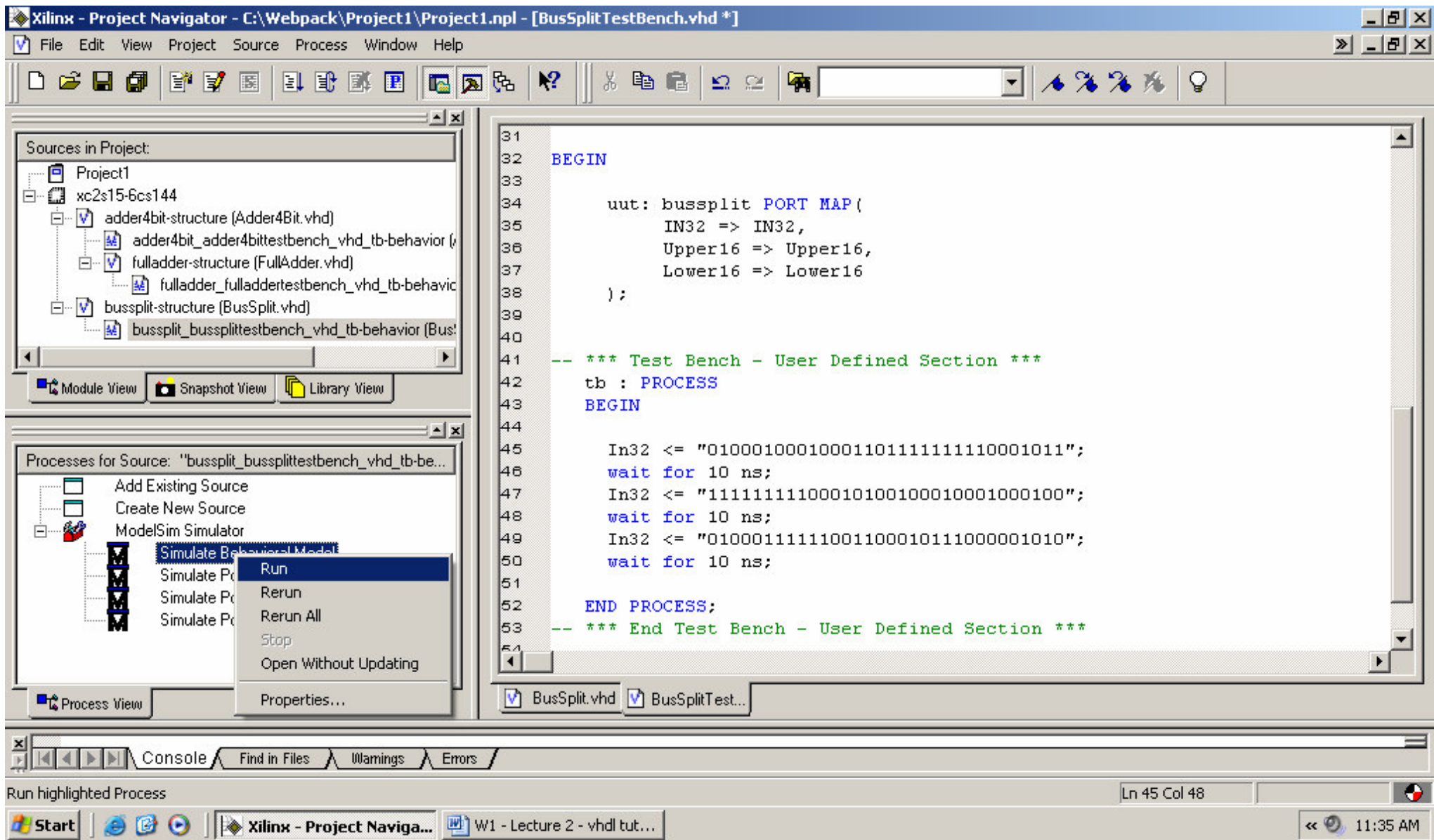
Note, you use “downto” as the keyword



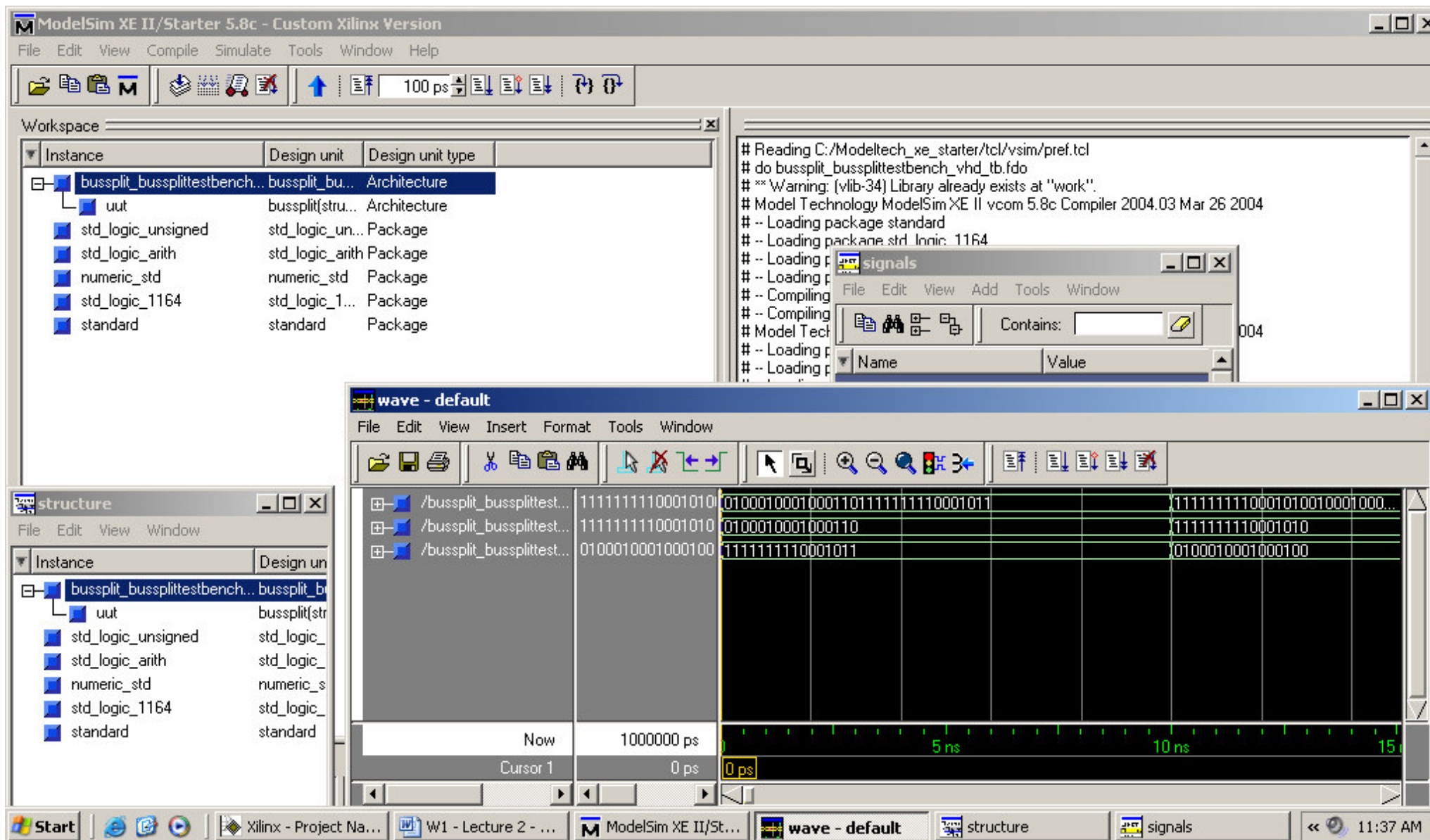
You can also check your syntax before you simulate or synthesize



Add the test bench for SplitBus module (same steps as before... some windows skipped).



Add the test bench code and run simulation



After zooming in/out and scroll to the left, you can see the result of the BusSplit module.

THE END OF TUTORIAL